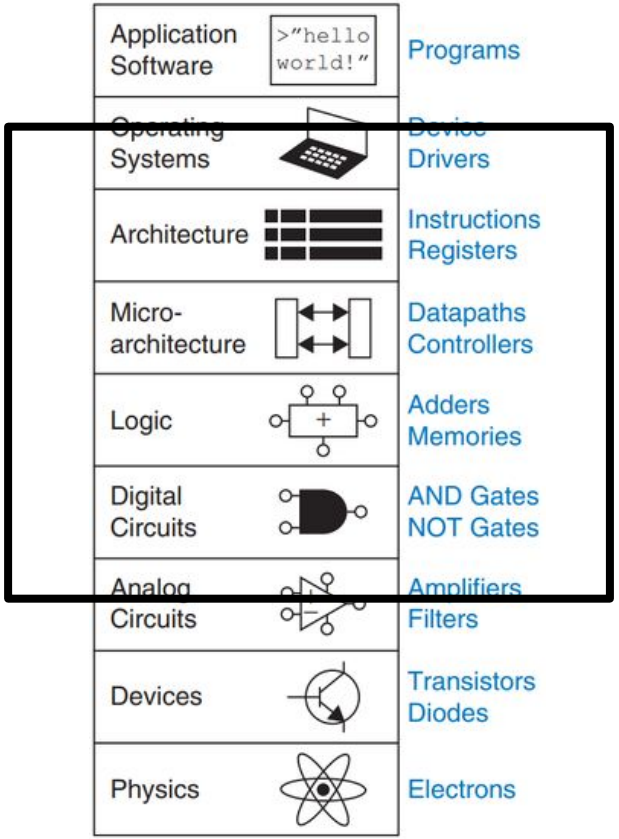
Aula 04: Fundamentos

VLSI + VDSM
Digital Fundamentals + ASIC Flow

RESIDÊNCIA EM MICROELETRÔNICA - POLO UFCG

Revisão

- **Aula Anterior (bottom-up)**
  - Modelos / MOSFET
  - Portas Lógicas / CMOS
  - Circuitos Combinacionais
  - Circuitos Sequenciais
  - Standard Cells / PDK
  - *PPA Budgets* (LIMITE)



VLSI

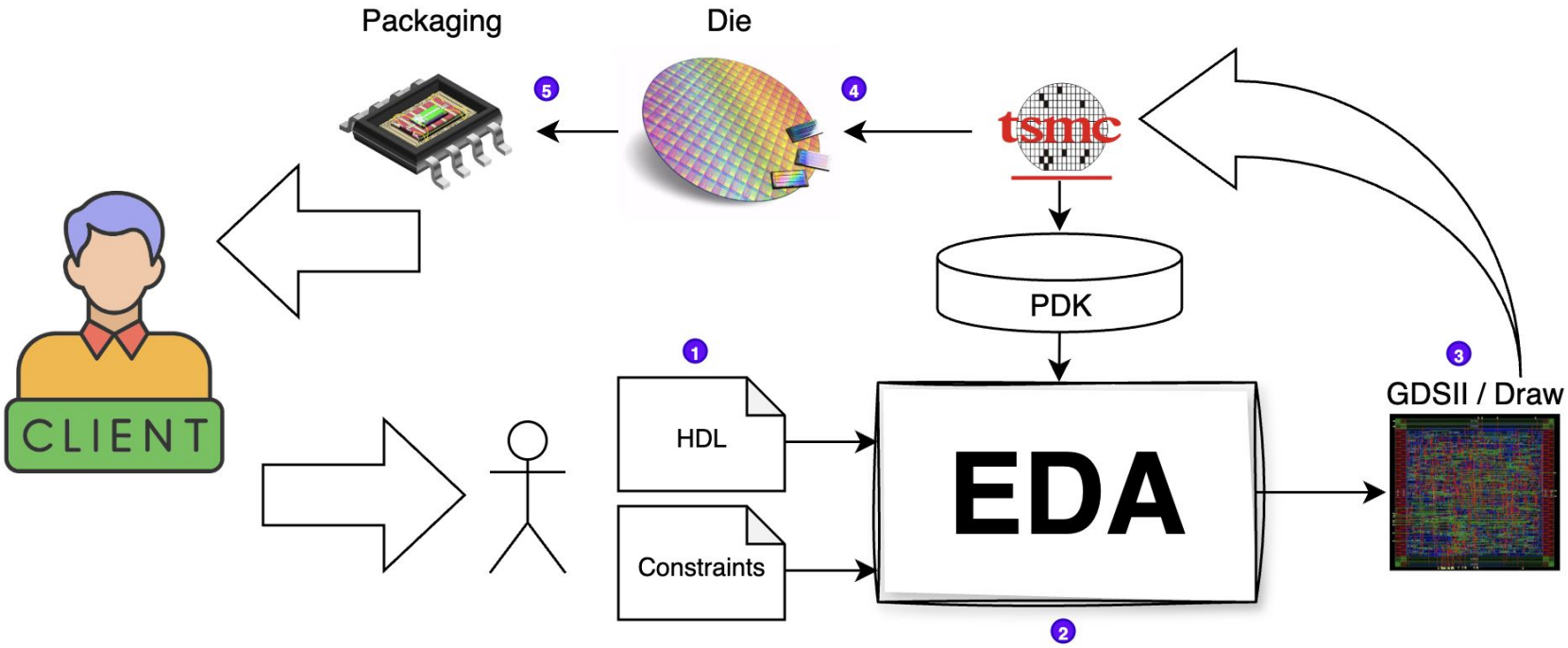
Agenda

- **Revisão aula anterior: VLSI Flow + VSDM (S4)**
  - Etapas do Fluxo e Ferramentas Envolvidas na Prática
  - Revisão do circuito FF + Comb + FF => VCS + Verdi
  - Noções de síntese na prática com DC Compiler + **Staic Timing Analysis**
- **Continução**
  - Front-end => Back-end Flow (MODELOS + PRECISOS)
  - Recapitulando principais conceitos => **Prova 02**
  - **Revisão da Prova 01**

AULA ANTERIOR

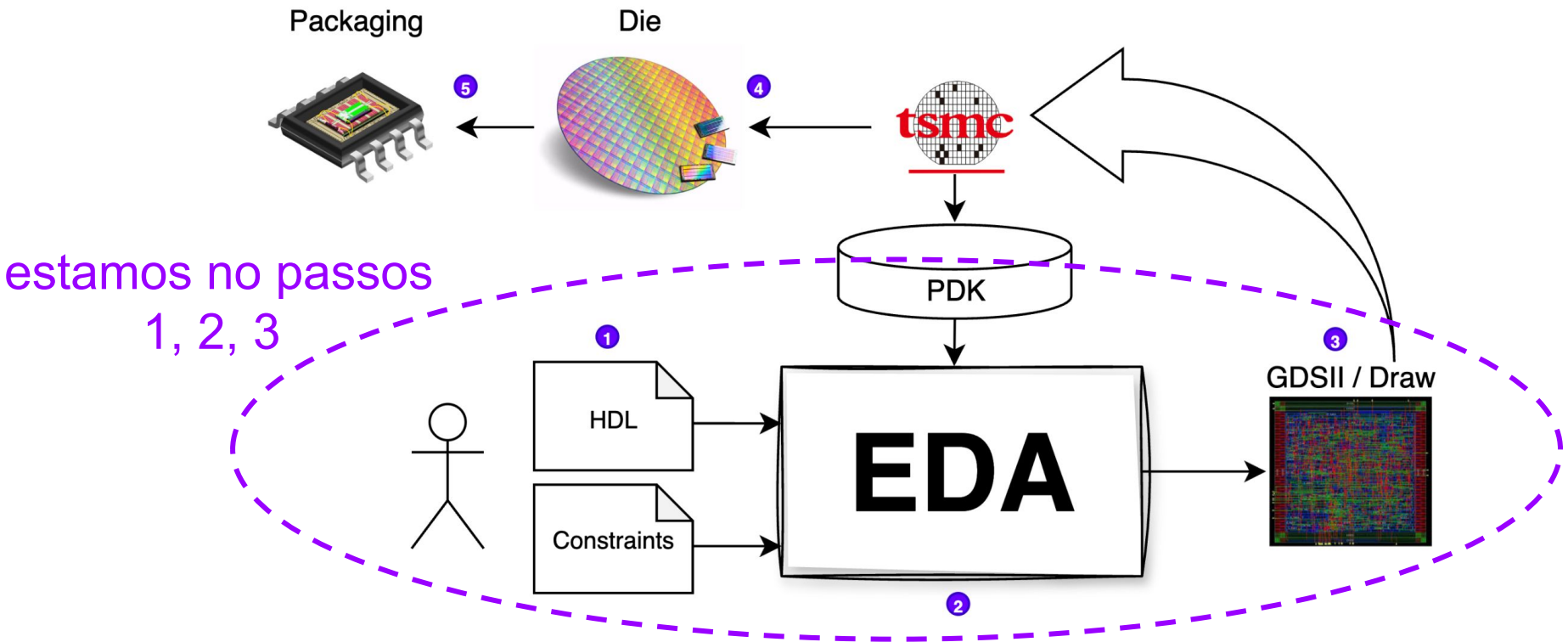
VLSI Flow

- Como tudo isso aparece no fluxo VLSI?



VLSI Flow

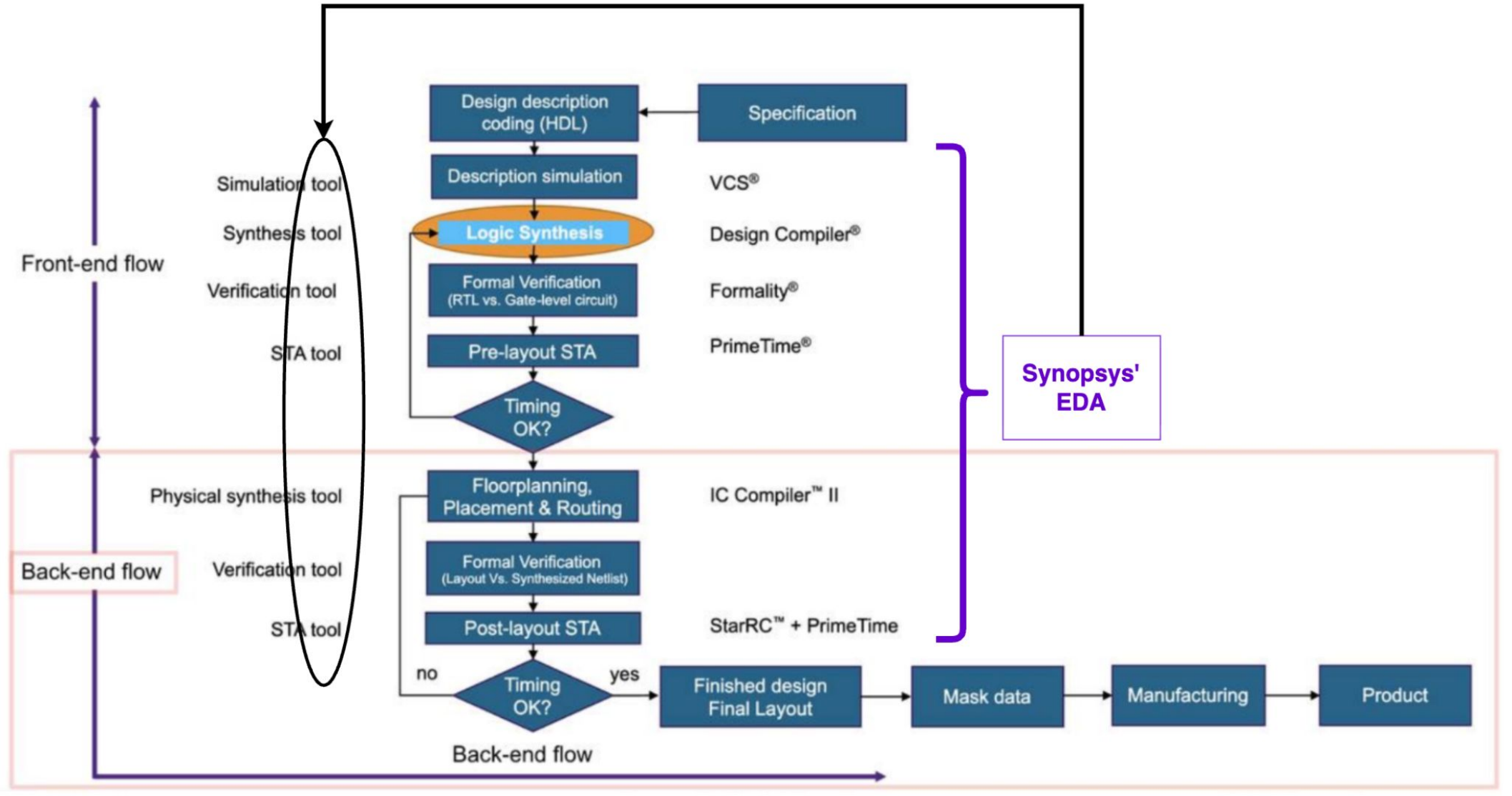
- Como tudo isso aparece no fluxo VLSI?



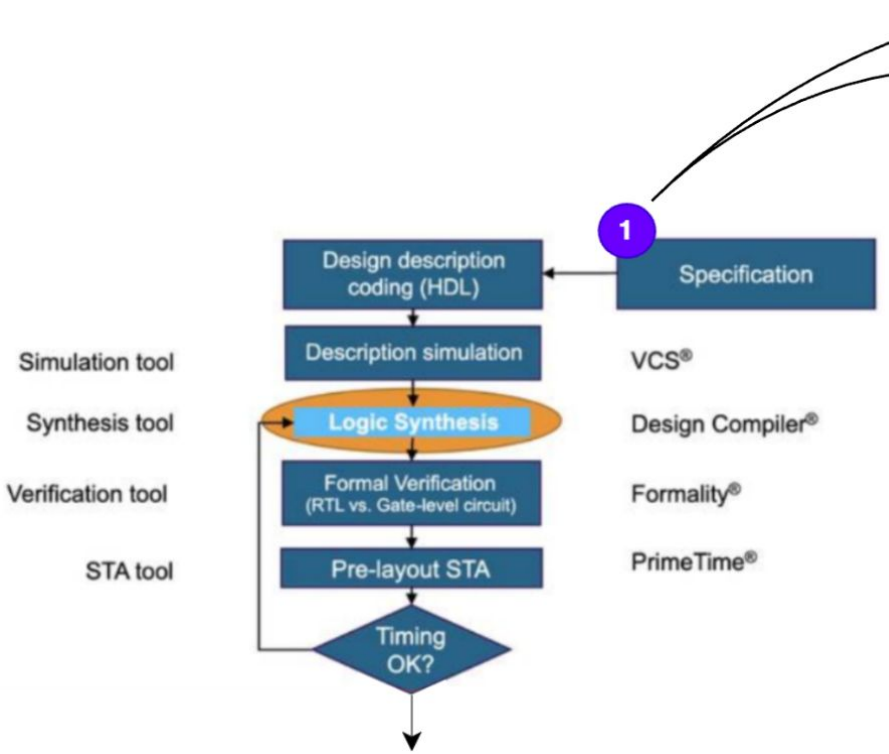
VLSI Flow

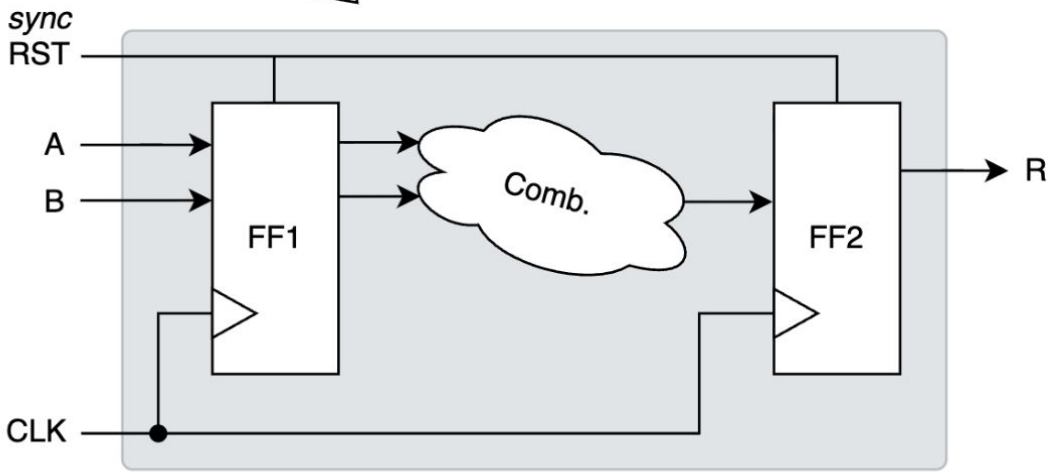
- Vamos entender o **fluxo front-end** na prática…

EDAs are tools to run the VLSI flow



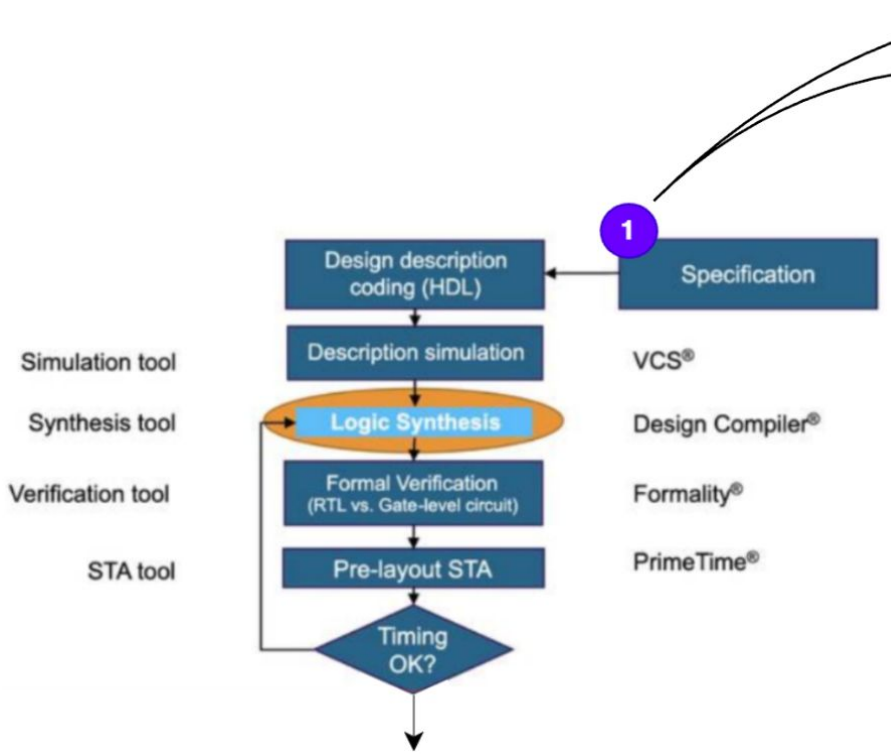
VLSI Flow

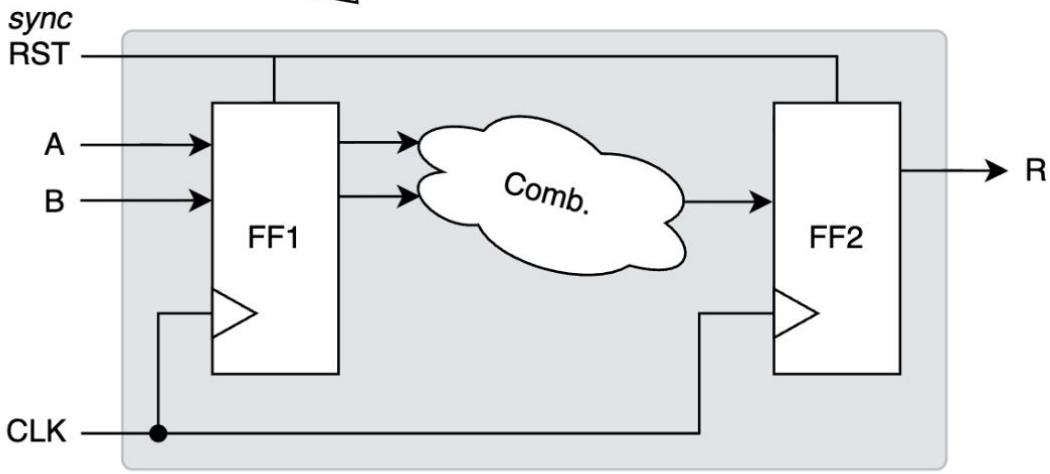




Q) Quantos flip-flops esse circuito tem?

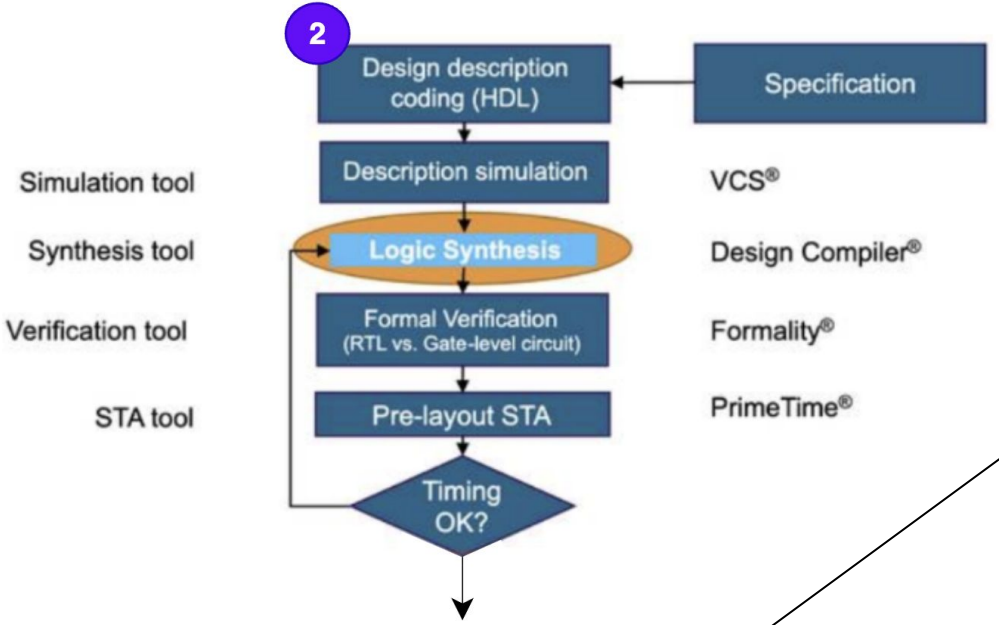
VLSI Flow

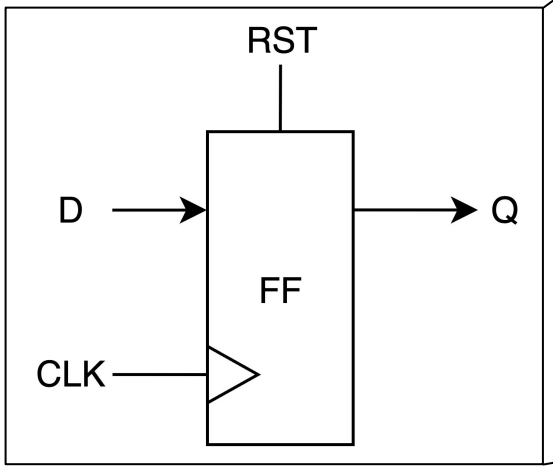




Q) Quantos flip-flops esse circuito tem? R=3FF

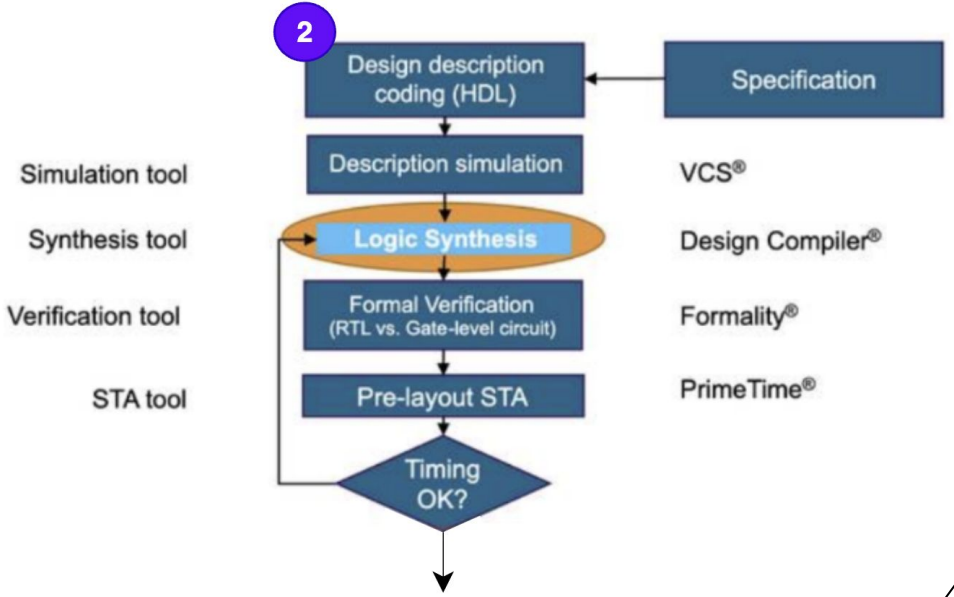
VLSI Flow

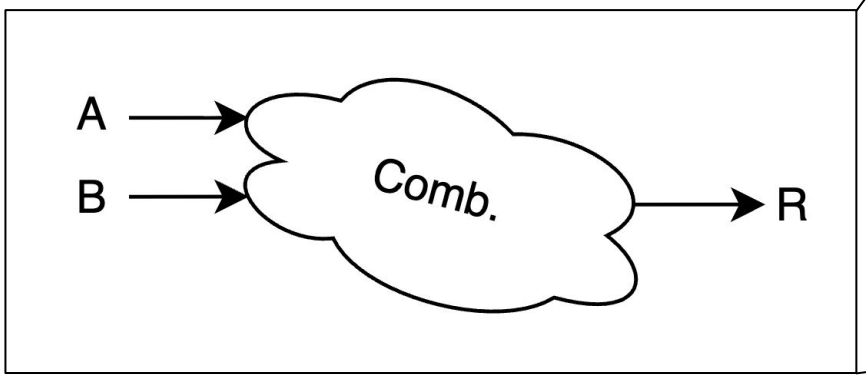





```systemverilog
module ff(
  input  logic CLK,
  input  logic RST,
  input  logic D,
  output logic Q
);

  always_ff @(posedge CLK) begin
    if(RST)
      Q <= 1'b0;
    else
      Q <= D;
  end

endmodule
```

VLSI Flow

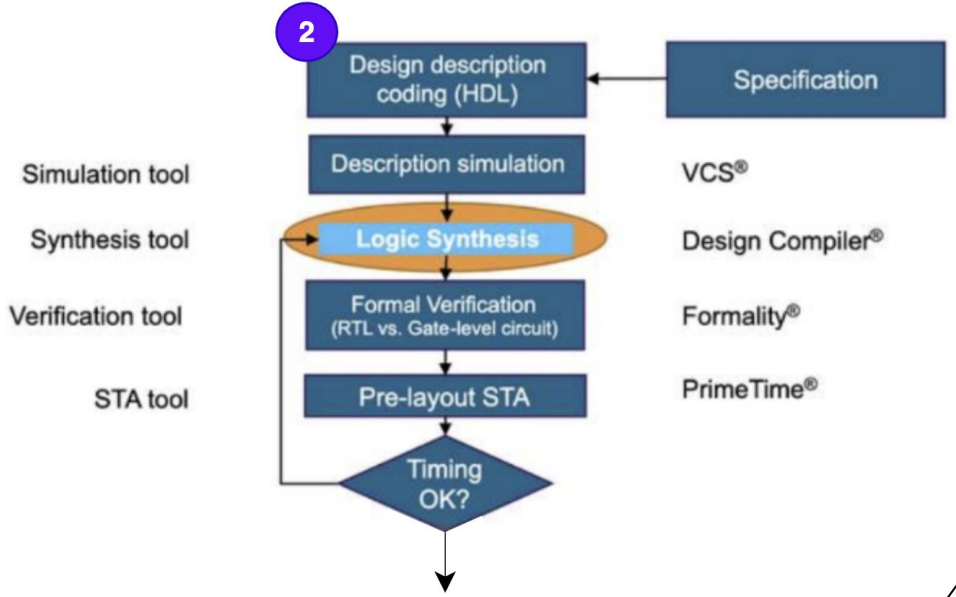


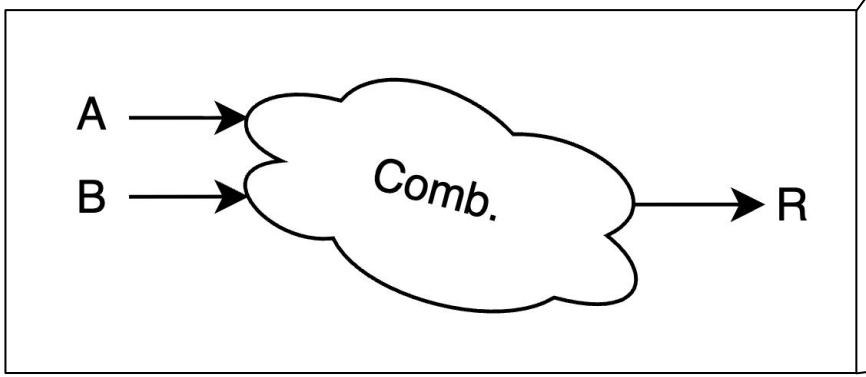



```
1  module comb(
2      input  logic A,
3      input  logic B,
4      output logic R
5  );
6
7      always_comb begin
8          R = A & B;
9      end
10
11 endmodule
```

Q) Qual circuito Comb. está modelando?

VLSI Flow






```
1  module comb(
2      input  logic A,
3      input  logic B,
4      output logic R
5  );
6
7      always_comb begin
8          R = A & B;
9      end
10
11 endmodule
```

Q) Qual circuito Comb. está modelando? R: AND

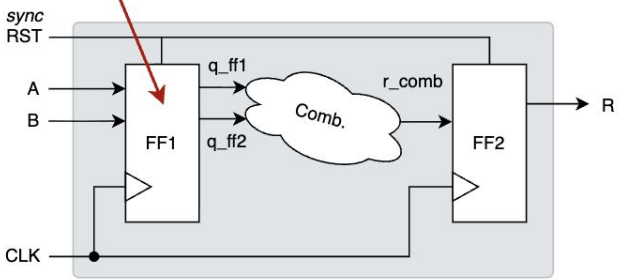
MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(
2    input  logic CLK,
3    input  logic RST,
4    input  logic A,
5    input  logic B,
6    output logic R
7  );
8    // "wires""
9    logic q_ff1, q_ff2, r_comb;
10
11   // FF input 1
12   ff uu_ff_in1(
13     .CLK(CLK  ),
14     .RST(RST  ),
15     .D  (A    ),
16     .Q  (q_ff1)
17   );
18
19   // FF input 2
20   ff uu_ff_in2(
21     .CLK(CLK  ),
22     .RST(RST  ),
23     .D  (B    ),
24     .Q  (q_ff2)
25   );
26
27   // FF
28   comb uu_comb(
29     .A(q_ff1 ),
30     .B(q_ff2 ),
31     .R(r_comb)
32   );
33
34   ff uu_ff_out(
35     .CLK(CLK   ),
36     .RST(RST   ),
37     .D  (r_comb),
38     .Q  (R     )
39   );
40 endmodule
```

PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets
REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:
<nome_do_modulo> <apelido_do_modulo>



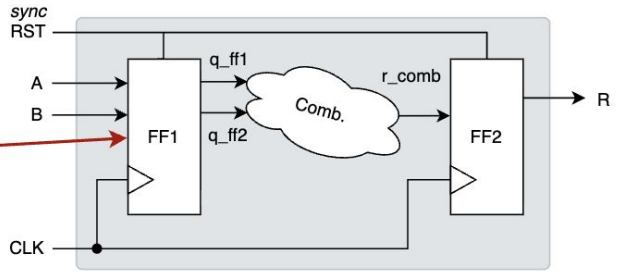
MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(
2    input  logic CLK,
3    input  logic RST,
4    input  logic A,
5    input  logic B,
6    output logic R
7  );
8    // "wires""
9    logic q_ff1, q_ff2, r_comb;
10
11   // FF input 1
12   ff uu_ff_in1(
13     .CLK(CLK  ),
14     .RST(RST  ),
15     .D  (A    ),
16     .Q  (q_ff1)
17   );
18
19   // FF input 2
20   ff uu_ff_in2(
21     .CLK(CLK  ),
22     .RST(RST  ),
23     .D  (B    ),
24     .Q  (q_ff2)
25   );
26
27   // FF
28   comb uu_comb(
29     .A(q_ff1 ),
30     .B(q_ff2 ),
31     .R(r_comb)
32   );
33
34   ff uu_ff_out(
35     .CLK(CLK   ),
36     .RST(RST   ),
37     .D  (r_comb),
38     .Q  (R     )
39   );
40 endmodule
```

PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets
REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:
<nome_do_modulo> <apelido_do_modulo>



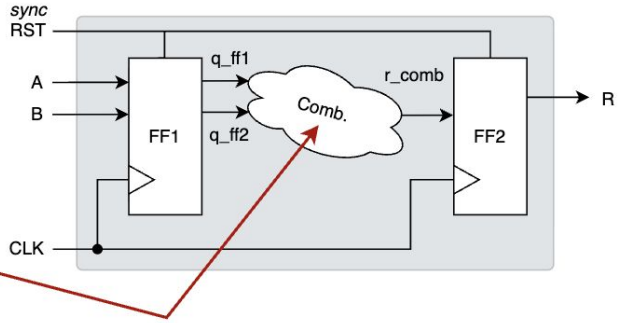
MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(
2    input  logic CLK,
3    input  logic RST,
4    input  logic A,
5    input  logic B,
6    output logic R
7  );
8    // "wires""
9    logic q_ff1, q_ff2, r_comb;
10
11   // FF input 1
12   ff uu_ff_in1(
13     .CLK(CLK  ),
14     .RST(RST  ),
15     .D  (A    ),
16     .Q  (q_ff1)
17   );
18
19   // FF input 2
20   ff uu_ff_in2(
21     .CLK(CLK  ),
22     .RST(RST  ),
23     .D  (B    ),
24     .Q  (q_ff2)
25   );
26
27   // FF
28   comb uu_comb(
29     .A(q_ff1 ),
30     .B(q_ff2 ),
31     .R(r_comb)
32   );
33
34   ff uu_ff_out(
35     .CLK(CLK   ),
36     .RST(RST   ),
37     .D  (r_comb),
38     .Q  (R     )
39   );
40 endmodule
```

PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets
REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:
<nome_do_modulo> <apelido_do_modulo>



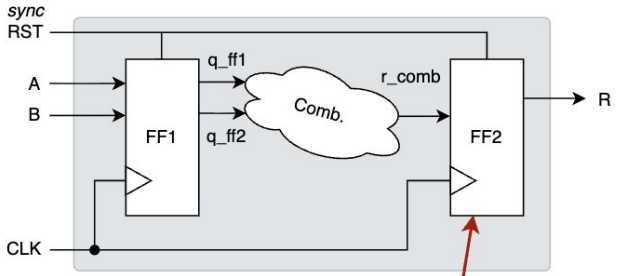
MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(
2    input  logic CLK,
3    input  logic RST,
4    input  logic A,
5    input  logic B,
6    output logic R
7  );
8    // "wires""
9    logic q_ff1, q_ff2, r_comb;
10
11   // FF input 1
12   ff uu_ff_in1(
13     .CLK(CLK  ),
14     .RST(RST  ),
15     .D  (A    ),
16     .Q  (q_ff1)
17   );
18
19   // FF input 2
20   ff uu_ff_in2(
21     .CLK(CLK  ),
22     .RST(RST  ),
23     .D  (B    ),
24     .Q  (q_ff2)
25   );
26
27   // FF
28   comb uu_comb(
29     .A(q_ff1 ),
30     .B(q_ff2 ),
31     .R(r_comb)
32   );
33
34   ff uu_ff_out(
35     .CLK(CLK   ),
36     .RST(RST   ),
37     .D  (r_comb),
38     .Q  (R     )
39   );
40 endmodule
```

PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets
REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:
<nome_do_modulo> <apelido_do_modulo>



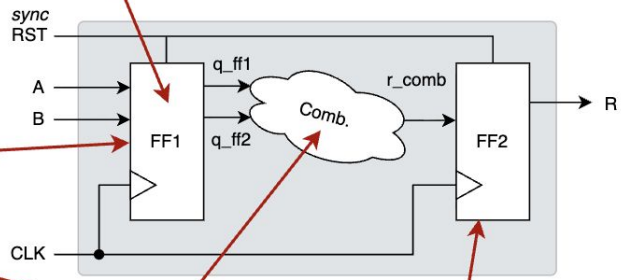
MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(
2    input  logic CLK,
3    input  logic RST,
4    input  logic A,
5    input  logic B,
6    output logic R
7  );
8    // "wires""
9    logic q_ff1, q_ff2, r_comb;
10
11   // FF input 1
12   ff uu_ff_in1(
13     .CLK(CLK  ),
14     .RST(RST  ),
15     .D  (A    ),
16     .Q  (q_ff1)
17   );
18
19   // FF input 2
20   ff uu_ff_in2(
21     .CLK(CLK  ),
22     .RST(RST  ),
23     .D  (B    ),
24     .Q  (q_ff2)
25   );
26
27   // FF
28   comb uu_comb(
29     .A(q_ff1 ),
30     .B(q_ff2 ),
31     .R(r_comb)
32   );
33
34   ff uu_ff_out(
35     .CLK(CLK   ),
36     .RST(RST   ),
37     .D  (r_comb),
38     .Q  (R     )
39   );
40 endmodule
```

PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets
REGS: variáveis internas que funcionam como registros são chamados de reg

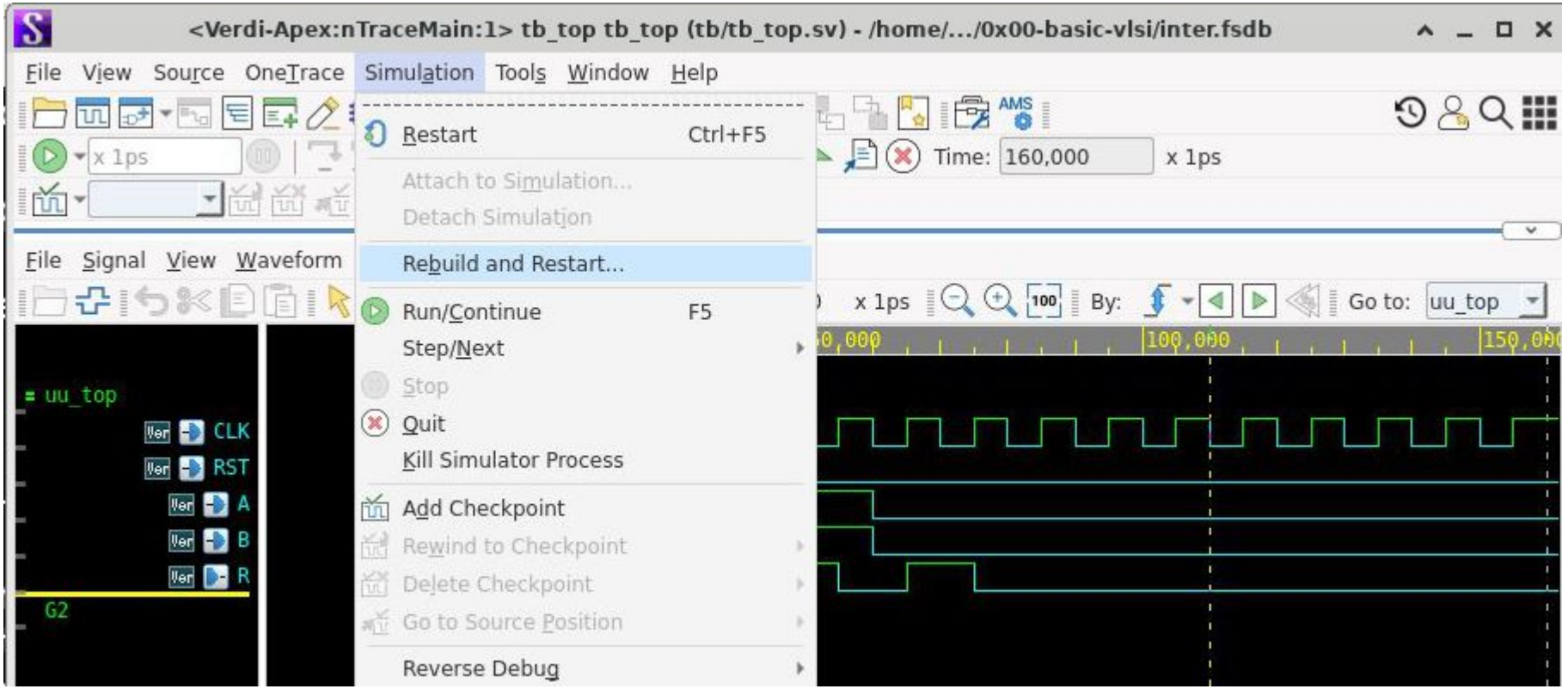
INSTANCIA DE MÓDULOS:
<nome_do_modulo> <apelido_do_modulo>



VCS + Verdi (prática ao vivo)

- Apresentar ambiente RTL + TB + Circuito
- VCS: Simulação apenas terminal
- Verdi: Inspeção esquemático gerado
- Verdi: Inspeção de formas de onda

+1 Verdi tip

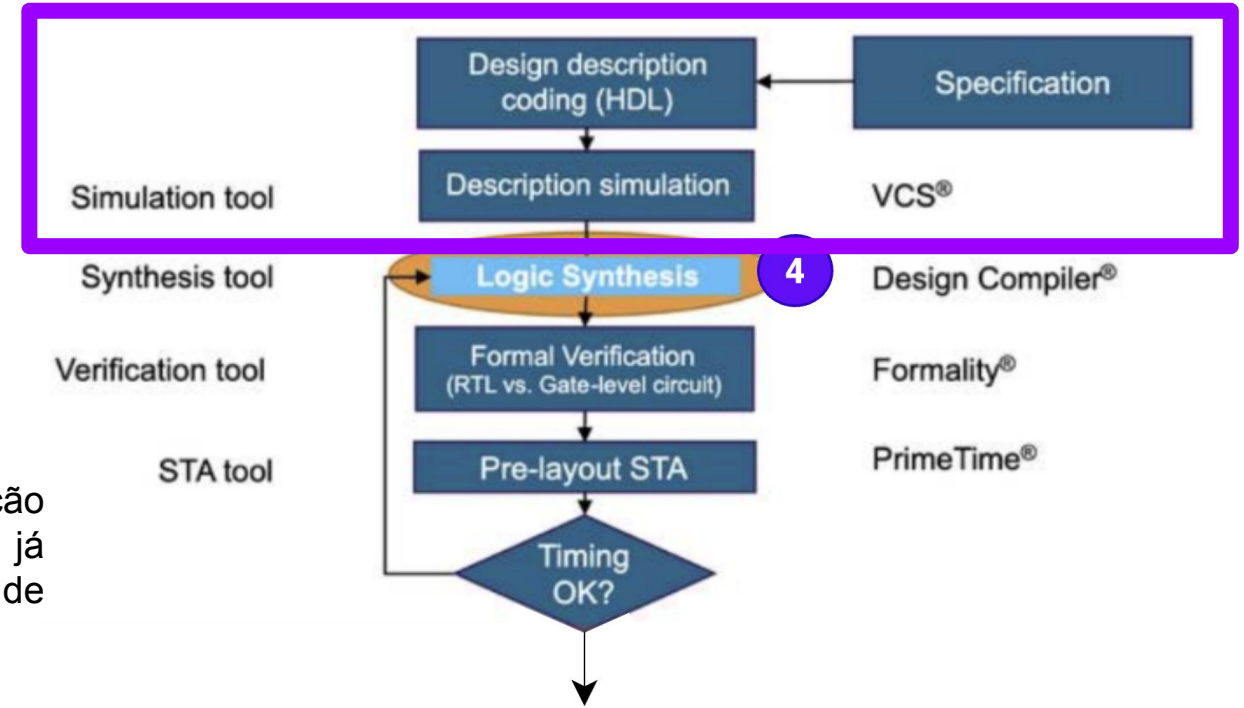

<Verdi-Apex:nTraceMain:1> tb_top tb_top (tb/tb_top.sv) - /home/.../0x00-basic-vlsi/inter.fsdb
File View Source OneTrace Simulation Tools Window Help
Restart Ctrl+F5
Attach to Simulation...
Detach Simulation
Rebuild and Restart...
Run/Continue F5
Step/Next
Stop
Quit
Kill Simulator Process
Add Checkpoint
Rewind to Checkpoint
Delete Checkpoint
Go to Source Position
Reverse Debug
Time: 160,000
x 1ps
File Signal View Waveform
Go to: uu_top
uu_top
CLK
RST
A
B
R
G2

Retomando de onde paramos no fluxo…

VLSI Flow

- **Descrição HDL** ✅
- **Testbench** ✅
- **Síntese** 🎯

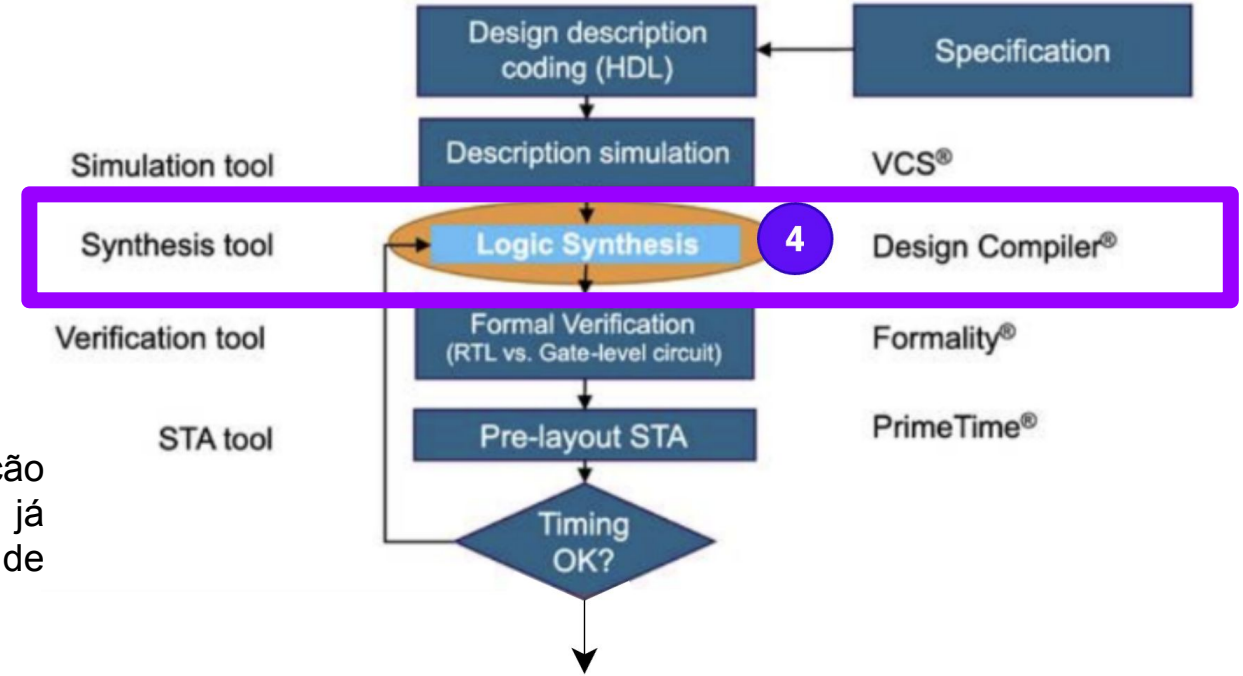
**síntese pode começar antes da finalização da verificação (TB); releases do circuito já iniciam síntese - adiantar o trabalho de *scripting* e checar por *bugs* grosseiros.



VLSI Flow

- **Descrição HDL** ✅
- **Testbench** ✅
- **Síntese** 🎯

**síntese pode começar antes da finalização da verificação (TB); releases do circuito já iniciam síntese - adiantar o trabalho de *scripting* e checar por *bugs* grosseiros.

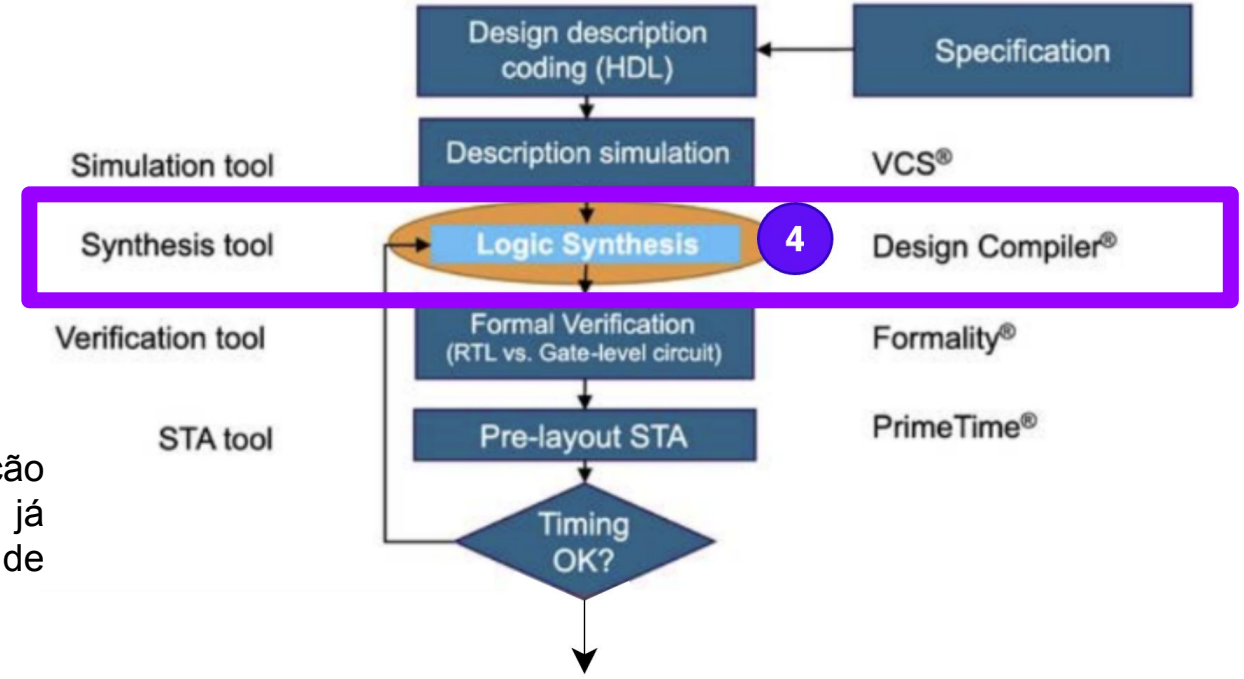


Q) O que a síntese faz e ela depende do que?

VLSI Flow

- **Descrição HDL** ✅
- **Testbench** ✅
- **Síntese** 🎯

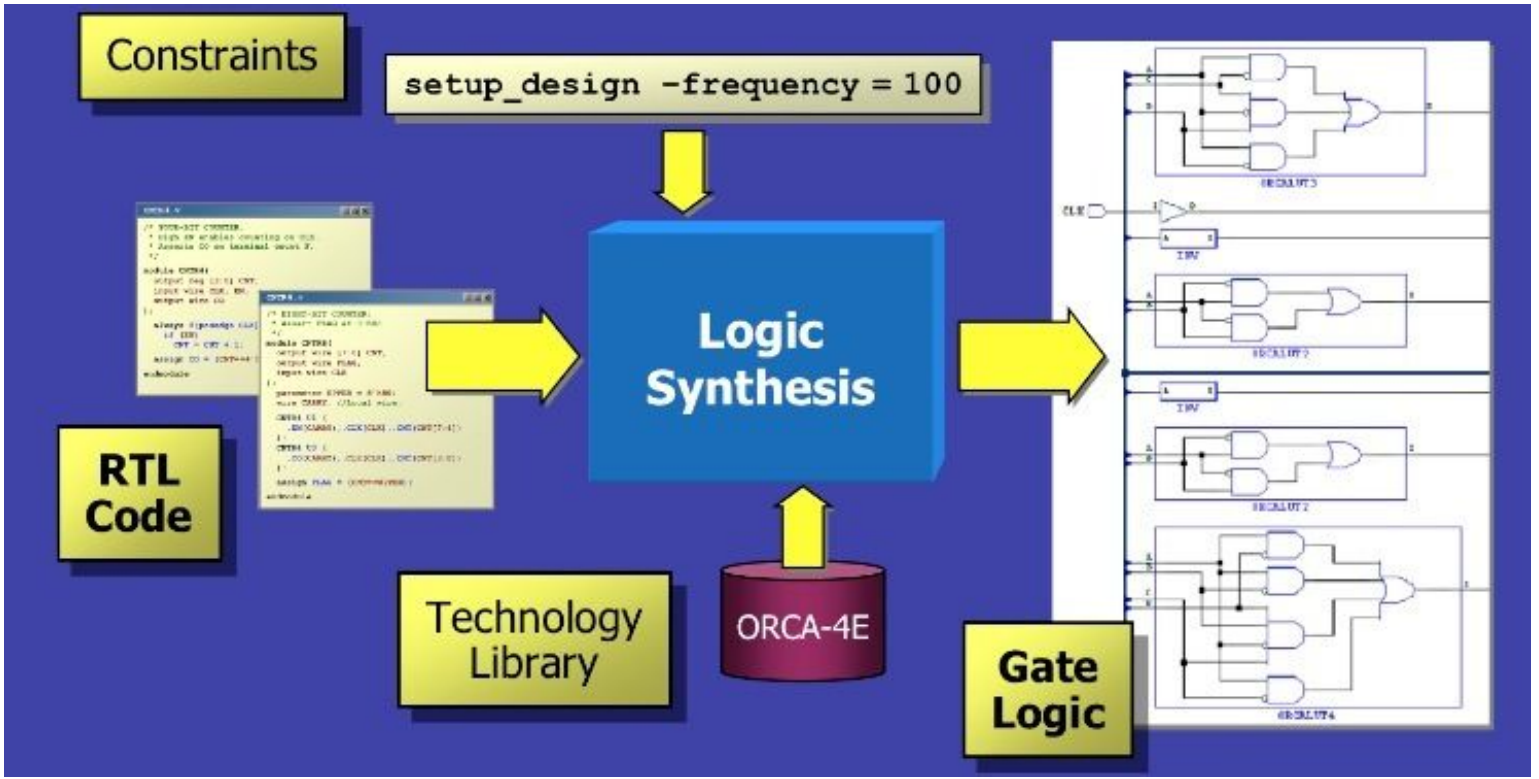
**síntese pode começar antes da finalização da verificação (TB); releases do circuito já iniciam síntese - adiantar o trabalho de *scripting* e checar por *bugs* grosseiros.



Q) O que a síntese faz e ela depende do que?

R: Transforma RTL em netlist lógica; Depende do PDK + Constraints

VLSI Flow


Constraints
setup_design -frequency = 100
RTL Code
Logic Synthesis
Technology Library
ORCA-4E
Gate Logic

VLSI Flow

- **RTL já temos** ✅
- **Constraints** 🎯
- **PDK** 🎯

VLSI Flow

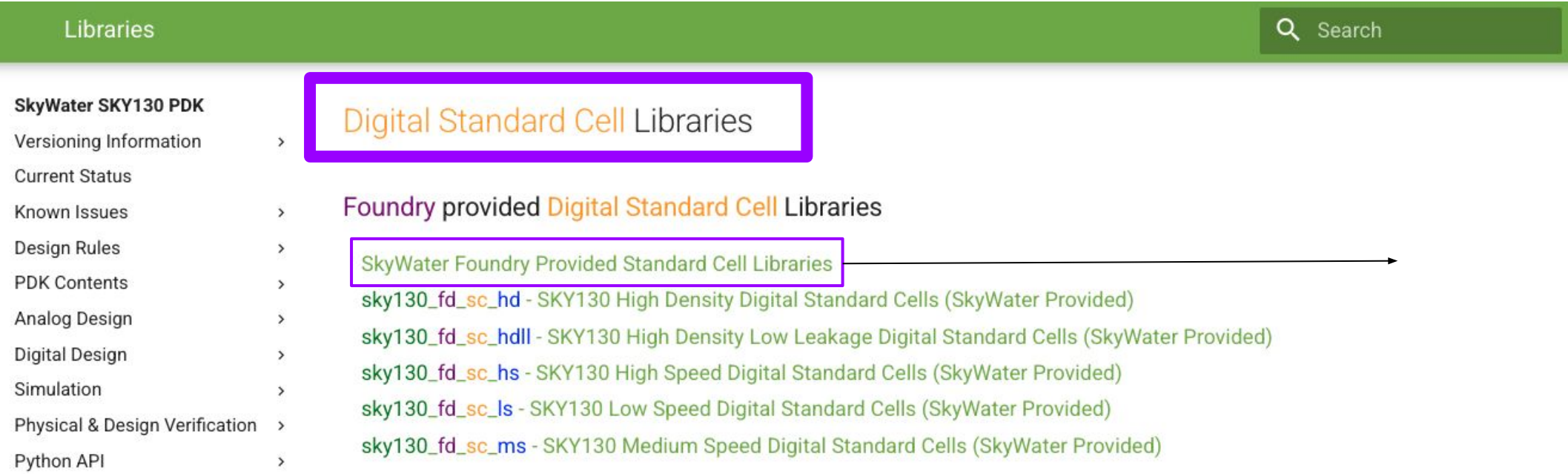
- **RTL já temos ✅**
- **Constraints 🎯**
- **PDK 🎯**

Vamos falar um pouco + de PDK...

VLSI Flow

- **PDK Skywater (OpenSource)**

https://skywater-pdk.readthedocs.io/en/main/contents/libraries.html#foundry-provided-digital-standard-cell-libraries



VLSI Flow

https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html

- **PDK Skywater (OpenSource)**

SkyWater Foundry Provided Standard Cell Libraries

There are seven standard cell libraries provided directly by the SkyWater Technology foundry available for use on SKY130 designs, which differ in intended applications and come in three separate cell heights.

Libraries sky130_fd_sc_hs (high speed), sky130_fd_sc_ms (medium speed), sky130_fd_sc_ls (low speed), and sky130_fd_sc_lp (low power) are compatible in size, with a 0.48 x 3.33um site, equivalent to about 11 `met1` tracks.

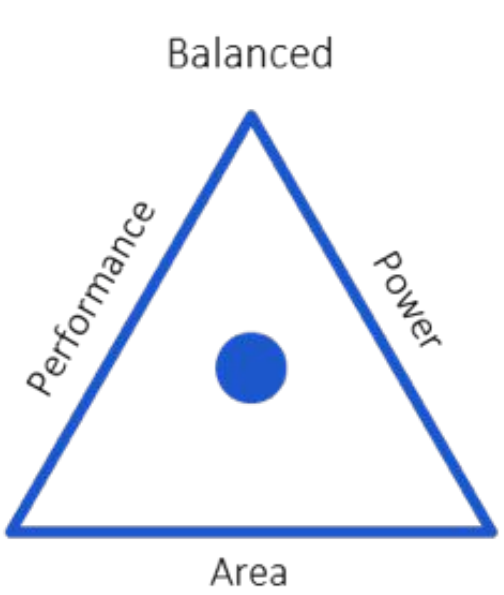
Libraries sky130_fd_sc_hd (high density) and sky130_fd_sc_hdll (high density, low leakage) contain standard cells that are smaller, utilizing a 0.46 x 2.72um site, equivalent to 9 `met1` tracks.

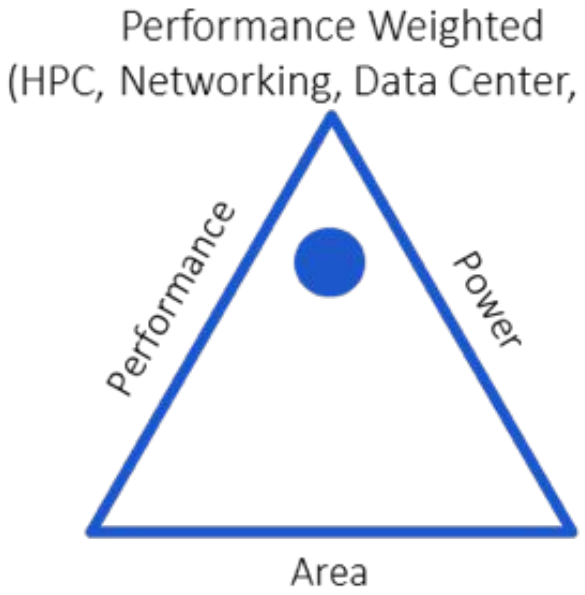
The sky130_fd_sc_hvl (high voltage) library contains 5V devices and utilizes a 0.48 x 4.07um site, or 14 `met1` tracks.

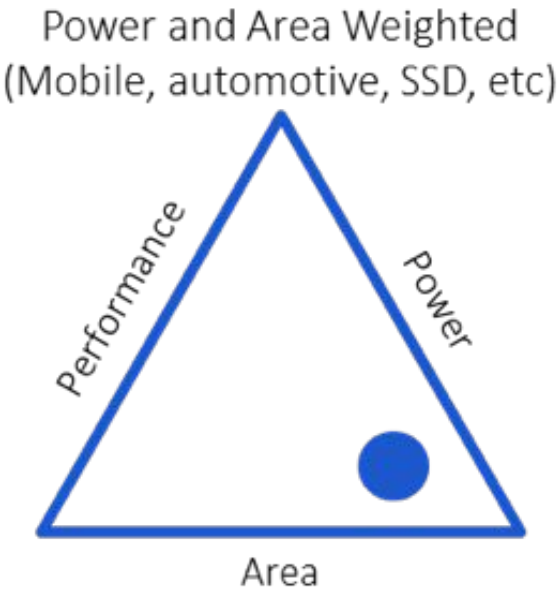
Coleção de células p/ diferentes tipos de necessidade

VLSI Flow

- **PPA**







	sky130_fd_sc_lp	sky130_fd_sc_ls	sky130_fd_sc_ms	sky130_fd_sc_hs	sky13
VDD	1.8	1.8	1.8	1.8	1.8
NMOS devices used	sky130_fd_pr__nfet_01v8	sky130_fd_pr__nfet_01v8	sky130_fd_pr__nfet_01v8_lvt	sky130_fd_pr__nfet_01v8_lvt	sky13
PMOS devices used	sky130_fd_pr__pfet_01v8_hvt	sky130_fd_pr__pfet_01v8_hvt	sky130_fd_pr__pfet_01v8	sky130_fd_pr__pfet_01v8_lvt	sky13
inverters, buffers	108	48	48	48	56
AND, OR, NAND, NOR	159	66	86	86	153
XOR, XNOR	16	12	12	12	8
AND-OR-INV, OR-AND-INV	138	71	71	71	115
AND-OR, OR-AND	132	68	68	68	132
Adders, Comparators, Multiplexors	59	37	33	33	31

https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html

	sky130_fd_sc_lp	sky130_fd_sc_ls	sky130_fd_sc_ms	sky130_fd_sc_hs	sky13
VDD	1.8	1.8	1.8	1.8	1.8
NMOS devices used	sky130_fd_pr__nfet_01v8	sky130_fd_pr__nfet_01v8	sky130_fd_pr__nfet_01v8_lvt	sky130_fd_pr__nfet_01v8_lvt	sky13
PMOS devices used	sky130_fd_pr__pfet_01v8_hvt	sky130_fd_pr__pfet_01v8_hvt	sky130_fd_pr__pfet_01v8	sky130_fd_pr__pfet_01v8_lvt	sky13
inverters,	108	48	48	48	56
Latches and Flip-Flops	92	68	68	68	60
Custom power gating, bus cells	43	66	42	42	51
OR-AND-INV					
AND-OR, OR-AND	132	68	68	68	132
Adders, Comparators, Multiplexors	59	37	33	33	31

tem memória também* / tudo q um CI tem

https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html

VLSI Flow

Exemplo documentação:

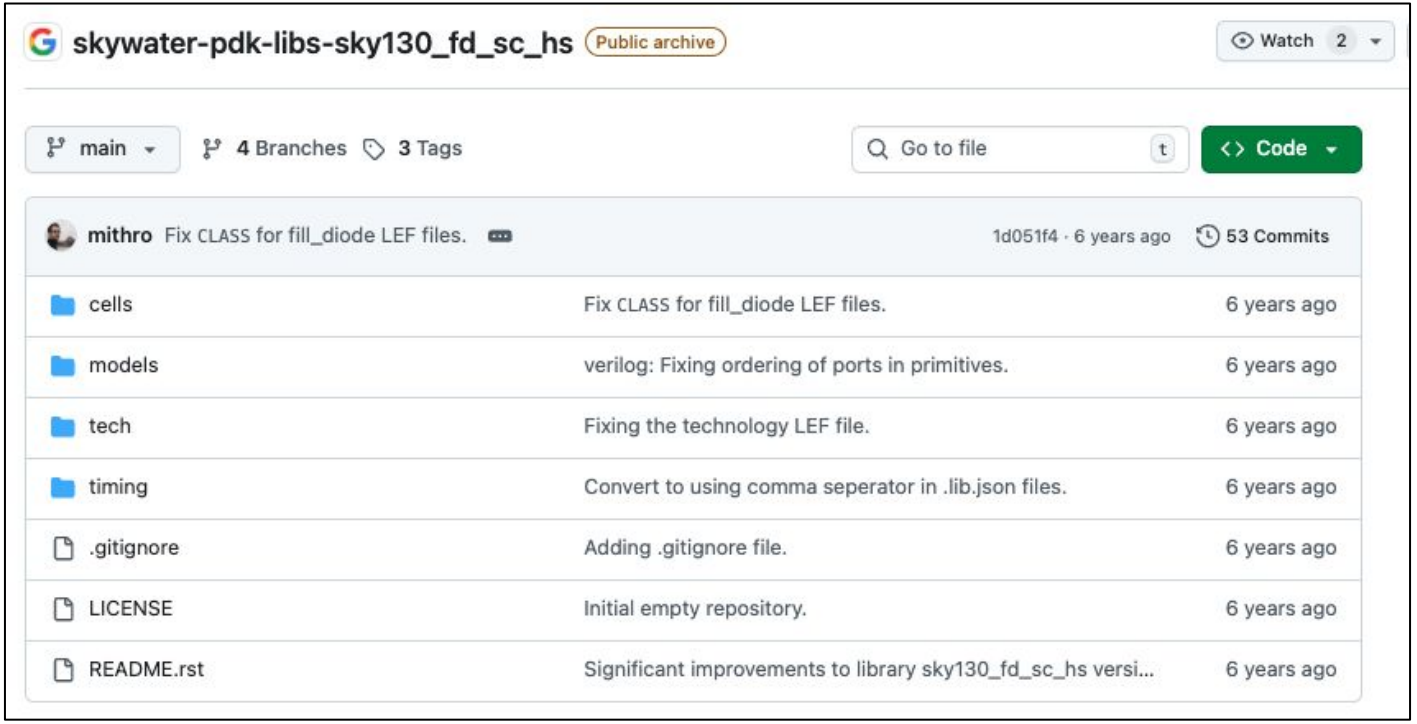
https://skywater-pdk.readthedocs.io/en/main/contents/libraries/sky130_fd_sc_hd/cells/mux2/README.html
https://skywater-pdk.readthedocs.io/en/main/contents/libraries/sky130_fd_sc_hd/cells/or2/README.html
https://skywater-pdk.readthedocs.io/en/main/contents/libraries/sky130_fd_sc_hd/cells/and2/README.html

Conclusão: PDK tem todos os ingredientes p/ fabricação de um VLSI.

VLSI Flow

- **PDK Skywater (OpenSource)**

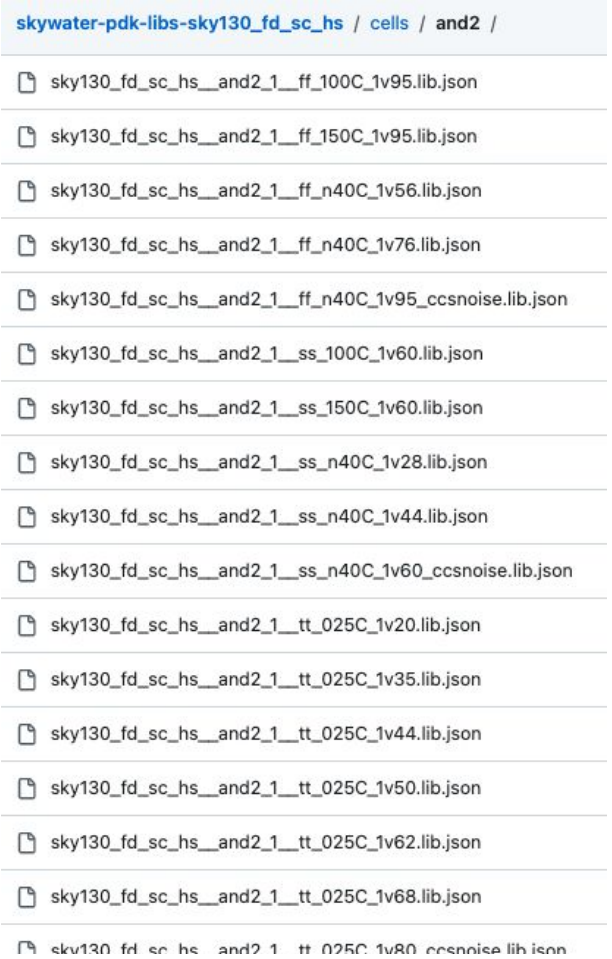
https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hs



VLSI Flow

- **PDK Skywater (OpenSource)**

https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hs/tree/main/cells/and2

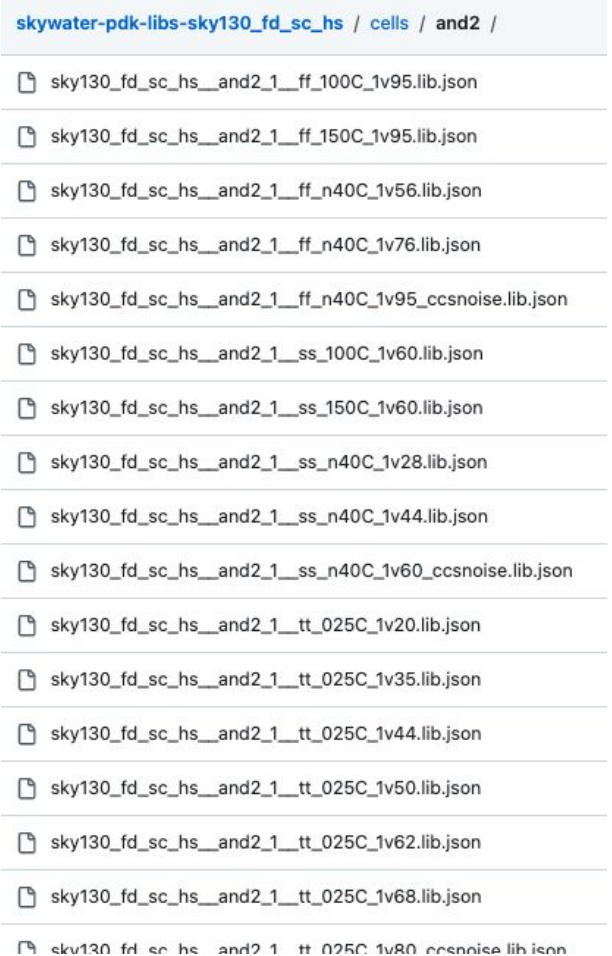


Q) Qual a diferença entre essas descrições?

VLSI Flow

- **PDK Skywater (OpenSource)**

https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hs/tree/main/cells/and2



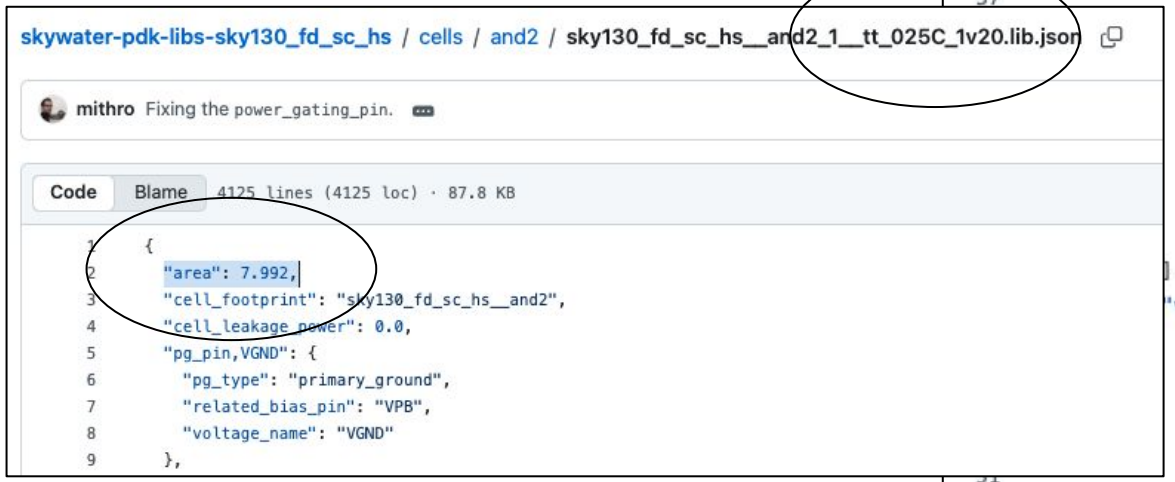
Q) Qual a diferença entre essas descrições?

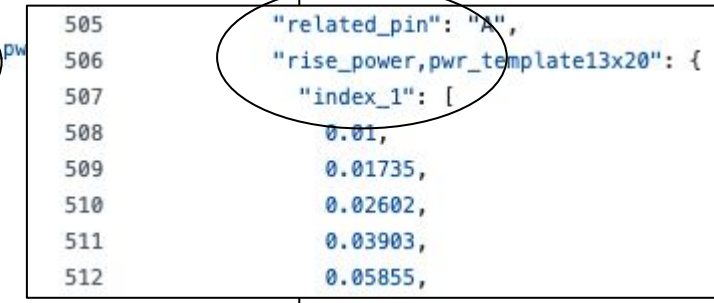
R: O processo e as condições de operação variam (PVT); O desempenho do circuito é afetado.

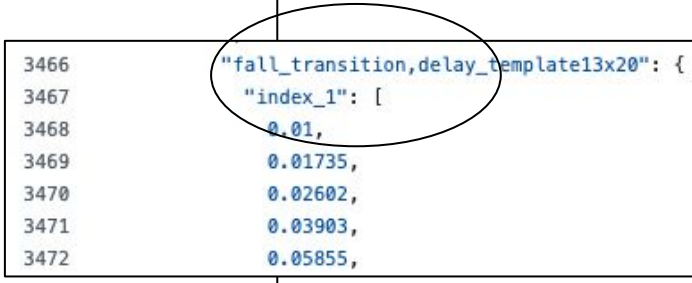
VLS

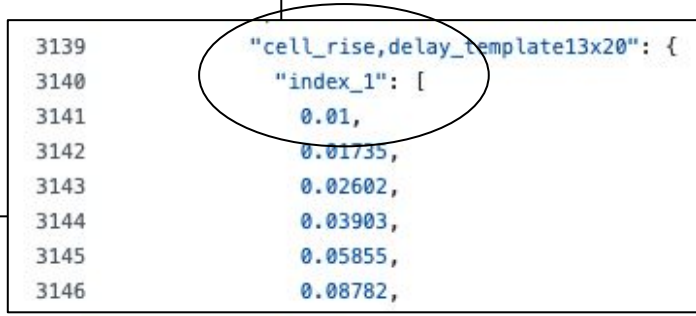
- **PDK Skywater (OpenSc**

https://github.com/google/skywater-pdk-libs-sky130_fd_s





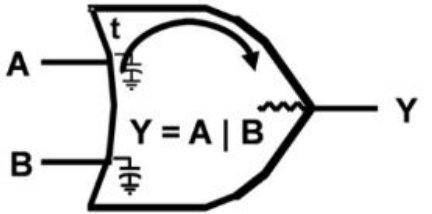


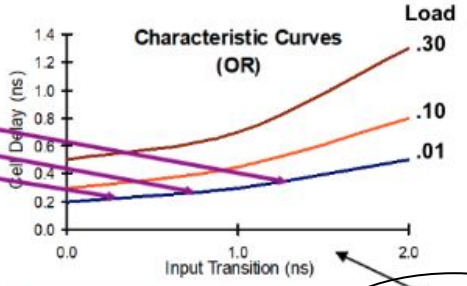


VLSI Flow (.lib + normal)

Example of a cell description in *.lib* format

```
cell ( OR2_4x ) {                                   <- Cell name
    area : 8.000 ;                                  <- Cell Area
    pin ( Y ) {
        direction : output ;
        timing ( ) {
            related_pin : "A" ;
            timing_sense : positive_unate ;
            rise_propagation (drive_3_table_1) {
                values ("0.2616, 0.2711, 0.2831,..)
            }
            rise_transition (drive_3_table_2) {
              values ("0.0223, 0.0254, ...)
                . . . .
        function : "(A | B)";                       <- Pin Y Functionality
        max_capacitance :  1.14810 ;                <- Design Rule Constraints
        min_capacitance :  0.00220 ;                   for Pin Y
    }
    pin ( A ) {
        direction : input ;
        capacitance : 0.012000;                     <- Electrical Characteristics
                . . . .                                of Pin A
```





Propagation and transition tables are "characterized" for a specific PVT corner, e.g. slowest or worst-case.

Logic libraries are usually supplied as binary *.db* files

VLSI Flow

- **Além disso, o .lib tem dois tipos de modelos**
  - **NLDM**: Nonlinear delay model (*less accurate*)
  - **CCS**: Composite current source (*more accurate*)

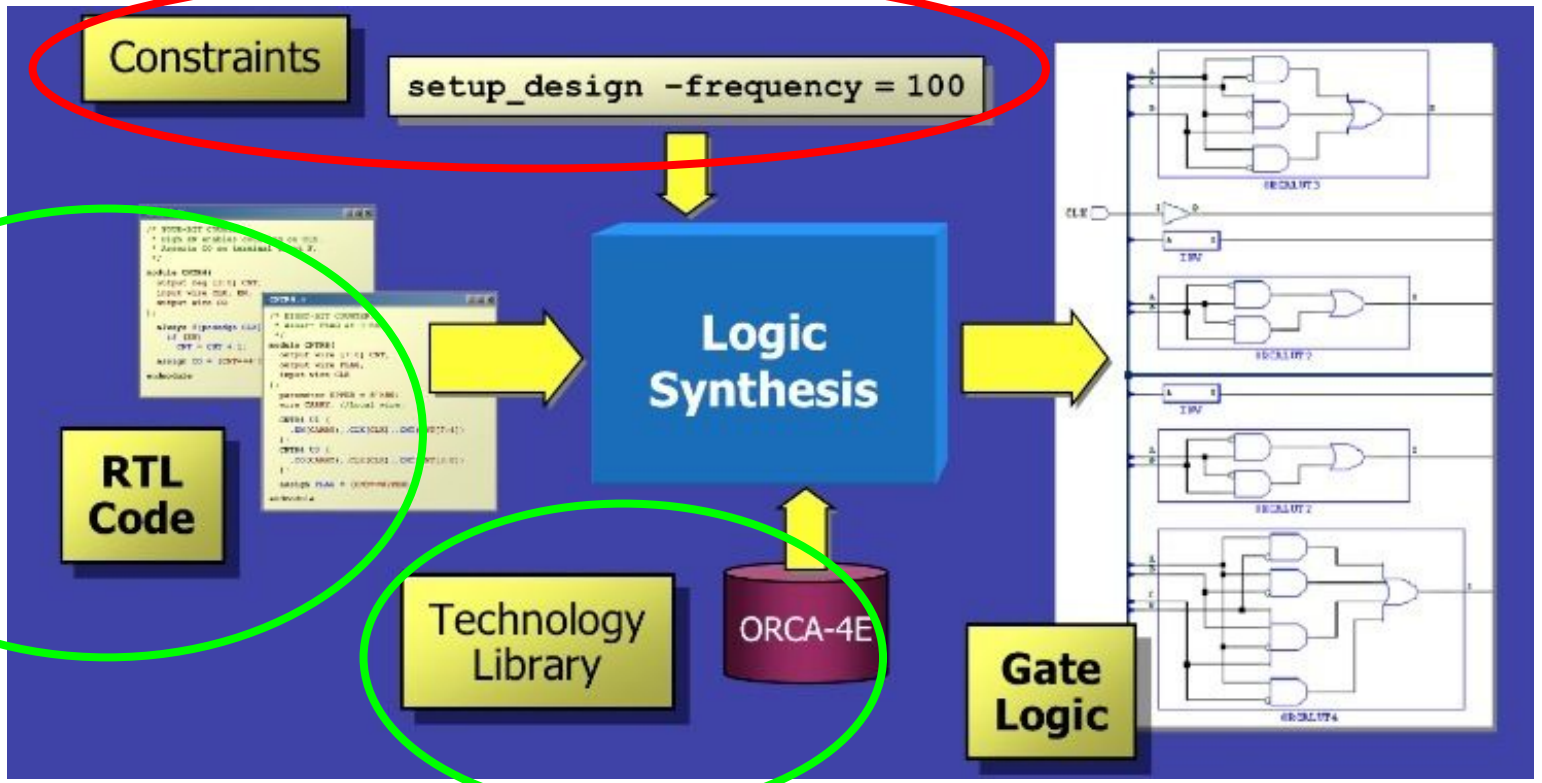
VLSI Flow

- **Conclusão**

  O KIT possui mais coisas, modelos de simulação das std-cells, LEF, DRC, doc, etc.

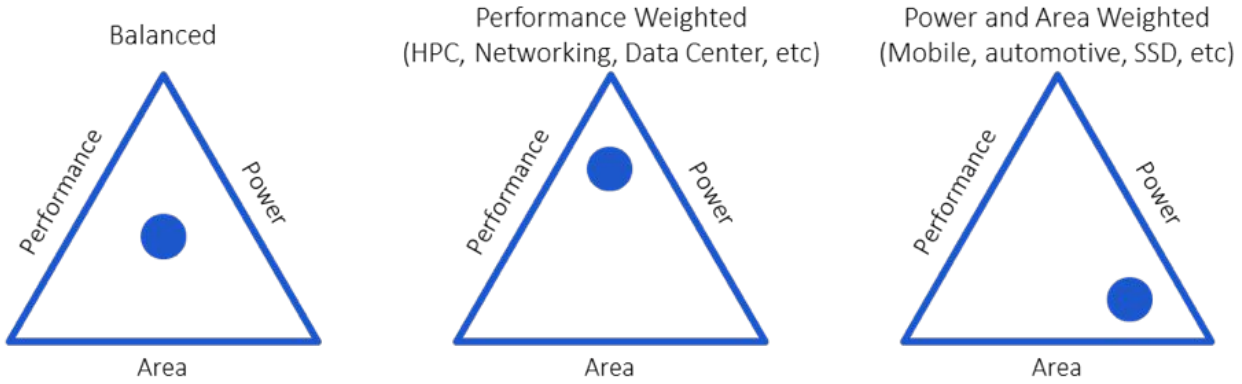
  - O **PDK**, que inclui bibliotecas de **standard cells** (células lógicas padronizadas e previamente caracterizadas em termos de área, potência e temporização) fornece as informações necessárias para **avaliar**, em diferentes níveis de abstração, o **PPA** (*power*, *performance* and *area*) do circuito integrado, considerando as variações de processo, tensão e temperatura (**PVT**).

VLSI Flow


Constraints
setup_design -frequency = 100
RTL Code
Logic Synthesis
Technology Library
ORCA-4E
Gate Logic

VLSI Flow

- **O próximo passo é definir *constraints*; isso GUIA a ferramenta a escolher, no PDK, quais células atendem aquele requisito não-funcional.**
- **O produto requer mais PERFORMANCE, MENOS CONSUMO DE BATERIA ou algo mais BALANCEADO?**



VLSI Flow

- A síntese depende de *constraints*;
- Constraints definem:
  - **PERIODO CLOCK (ou CLOCKS)**
  - **INCERTEZA DO CLOCK (ou CLOCKS)**
  - **RELAÇÃO ENTRE CLOCKS**
  - **SKEW, SLEW, JITTER**
  - **LATÊNCIA DE DADO DE ENTRADA**
  - **LATÊNCIA DE DADO DE SAÍDA**
  - **...**
- Isso tudo afeta como a ferramenta vai usar as células do PDK, já que existem várias, com diferentes característica…

VLSI Flow

- Na prática constraints são definidas assim (TCL)

dc_cons_setup.tcl

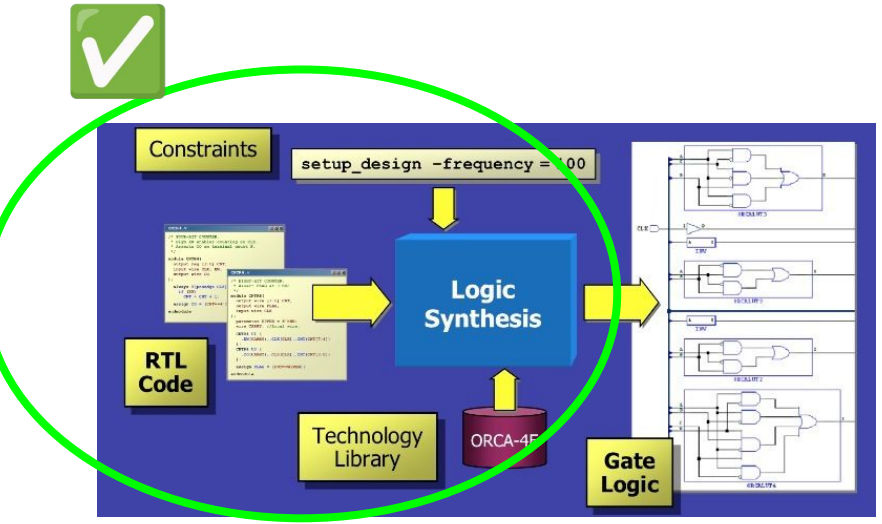
prj > 0x01-basic-vlsi-synth > dc-scripts > dc_cons_setup.tcl

```tcl
# ------------------------------------------------------------------------------
# Constraints Setup and Pre-Compiled Reports
# ------------------------------------------------------------------------------

# create clock
create_clock -period $clk_val [get_ports CLK] -name clk

# characteristics of clk
set_clock_uncertainty     -setup [expr $clk_val*0.1000] [get_clocks clk]
set_clock_transition      -max   [expr $clk_val*0.1000] [get_clocks clk]
set_clock_latency -source -max   [expr $clk_val*0.0500] [get_clocks clk]
set_clock_latency         -max   [expr $clk_val*0.0328] [get_clocks clk]

# input and output delay
set_input_delay  -max [expr $clk_val*0.4] -clock clk [remove_from_collection [all_inputs] [get_ports CLK]]
set_output_delay -max [expr $clk_val*0.5] -clock clk [all_outputs]

# output load
set_load -max 0.04 [all_outputs]

# input transition
set_input_transition -min [expr $clk_val*0.01] [remove_from_collection [all_inputs] [get_ports CLK]]
set_input_transition -max [expr $clk_val*0.10] [remove_from_collection [all_inputs] [get_ports CLK]]
```

VLSI Flow

- **Síntese**

Temos todos os ingredientes para dar mais um passo no fluxo…



VLSI Flow

- **Síntese**

As etapas do ferramental são essas:

```
if(high_bits == 2'b10)begin
  residue = state_table[i];
end
else begin
 residue = 16'h0000;
end
```

HDL Source (RTL)

Translate (HDL Compiler)

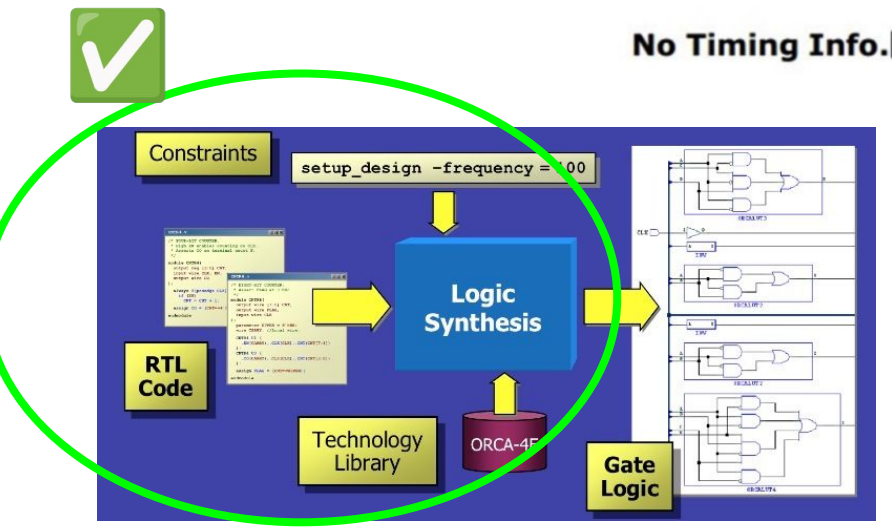
No Timing Info.

Generic Boolean GTECH

Optimize + Mapping (Design Compiler)

Timing Info.

Target Technology



VLSI Flow

- **Síntese**

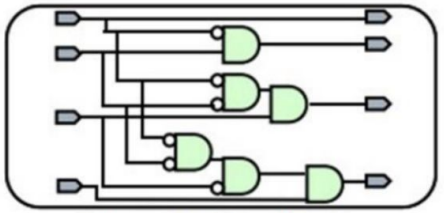
Etapa 1: translate
=> GTECH: **não depende** de PDK
Etapa 2: opt+mapping
=> **depende** de PDK
Vamos fazer isso na prática…

```
if(high_bits == 2'b10)begin
  residue = state_table[i];
end
else begin
 residue = 16'h0000;
end
```

HDL Source (RTL)

Translate (HDL Compiler)

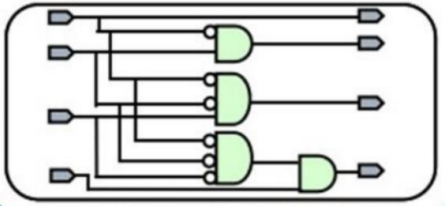
No Timing Info.



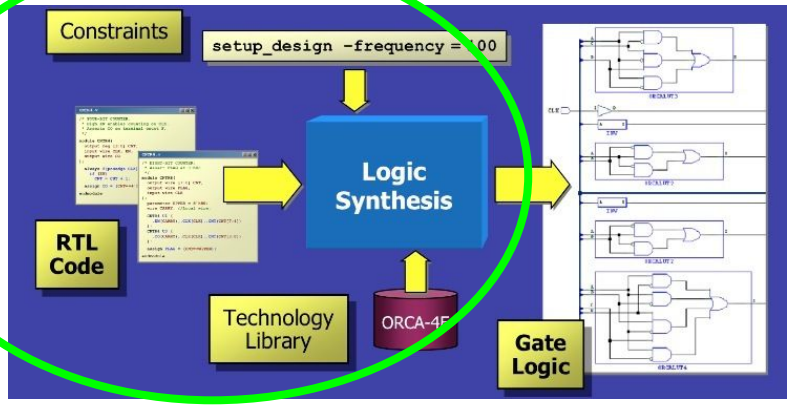
Generic Boolean
GTECH

Optimize + Mapping (Design Compiler)

Timing Info.



Target Technology



PDK + Constraints + Síntese (prática ao vivo)

- PDK .lib / .db (list)
- Setup constraints
- Results
- Increase clock freq or circuito

VLSI Flow / GTECH

ff.sv

prj > 0x00-basic-vlsi > rtl > ff.sv

```
module ff(
  input  logic CLK,
  input  logic RST,
  input  logic D,
  output logic Q
);

  always_ff @(posedge CLK) begin
    if(RST)
      Q <= 1'b0;
    else
      Q <= D;
  end

endmodule
```

comb.sv

prj > 0x00-basic-vlsi > rtl > comb.sv

```
module comb(
  input  logic A,
  input  logic B,
  output logic R
);

  always_comb begin
    R = A & B;
  end

endmodule
```

top.sv

prj > 0x00-basic-vlsi > rtl > top.sv

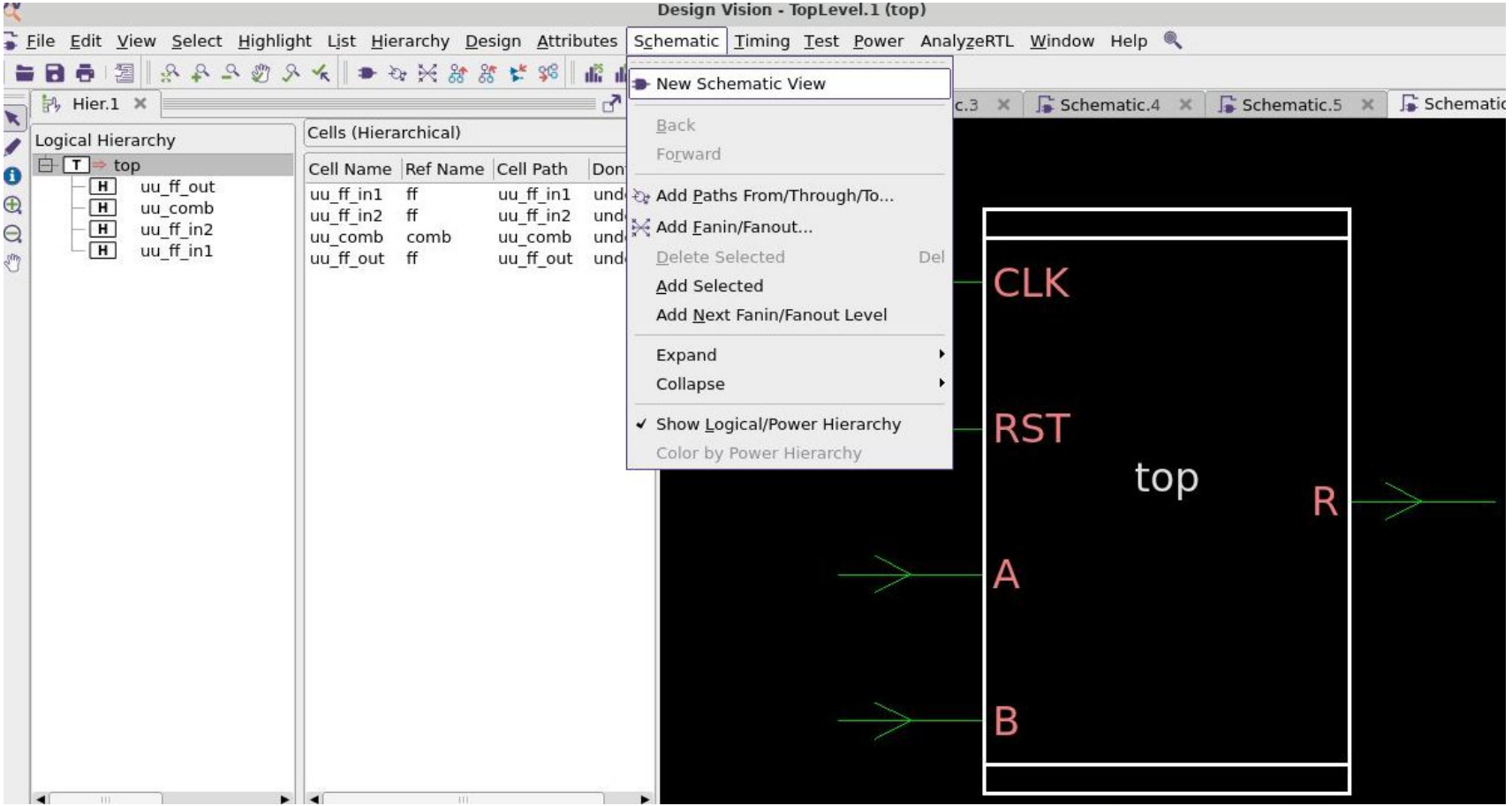
```
module top(
  input  logic CLK,
  input  logic RST,
  input  logic A,
  input  logic B,
  output logic R
);
  // "wires""
  logic q_ff1, q_ff2, r_comb;

  // FF input 1
  ff uu_ff_in1(
    .CLK(CLK  ),
    .RST(RST  ),
    .D  (A    ),
    .Q  (q_ff1)
  );

  // FF input 2
  ff uu_ff_in2(
    .CLK(CLK  ),
    .RST(RST  ),
    .D  (B    ),
    .Q  (q_ff2)
  );

  // FF
  comb uu_comb(
    .A(q_ff1 ),
    .B(q_ff2 ),
    .R(r_comb)
  );

  ff uu_ff_out(
    .CLK(CLK   ),
    .RST(RST   ),
    .D  (r_comb),
    .Q  (R     )
  );
endmodule
```

gtech_unmapped.v

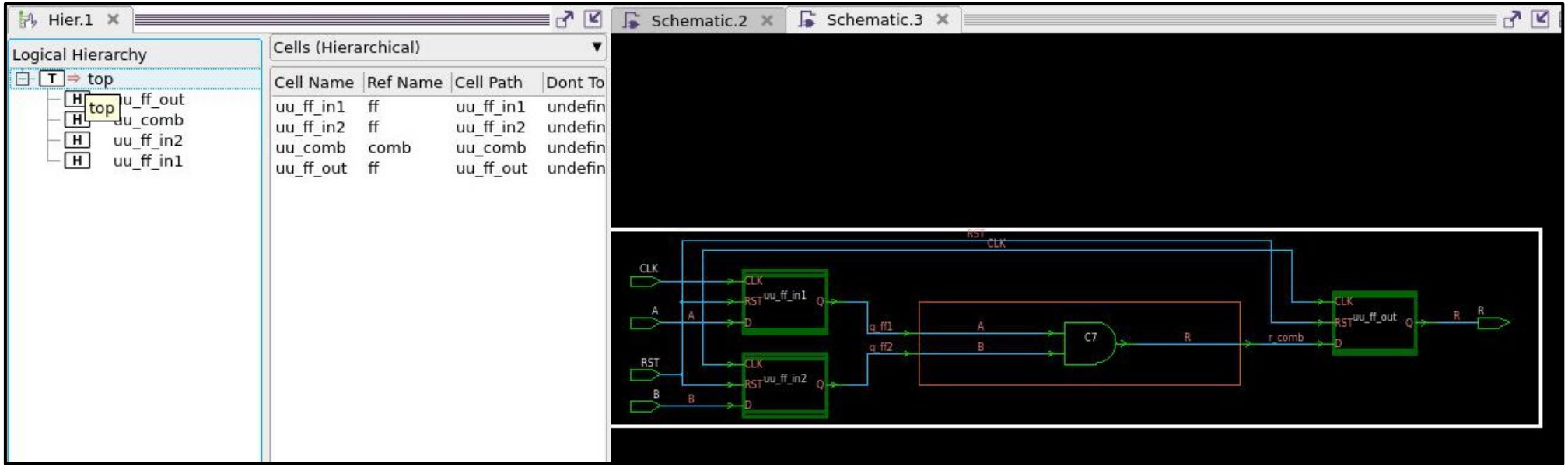
prj > 0x01-basic-vlsi-synth > dc-scripts > outputs_top > gtech_unmapped.v

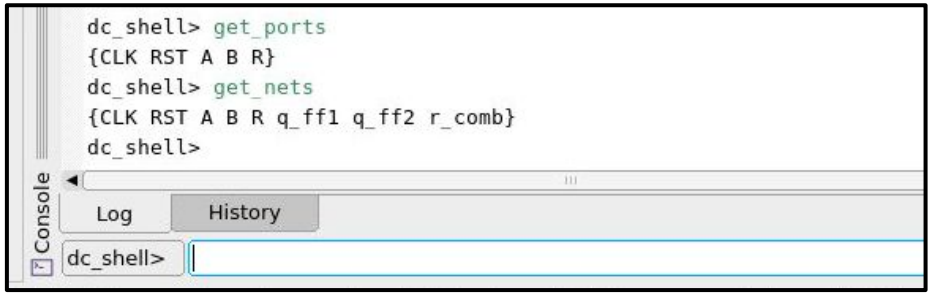
```
////////////////////////////////////////////////////////////
// Created by: Synopsys Design Compiler(R)
// Version   : X-2025.06-SP4
// Date      : Mon Apr  6 17:33:01 2026
////////////////////////////////////////////////////////////


module ff ( CLK, RST, D, Q );
  input CLK, RST, D;
  output Q;
  wire   N0, N1, N2, N3;

  \**SEQGEN**  Q_reg ( .clear(1'b0), .preset(1'b0), .next_state(N3),
        .clocked_on(CLK), .data_in(1'b0), .enable(1'b0), .Q(Q), .synch_clear(
        1'b0), .synch_preset(1'b0), .synch_toggle(1'b0), .synch_enable(1'b1)
        );
  SELECT_OP C11 ( .DATA1(1'b0), .DATA2(D), .CONTROL1(N0), .CONTROL2(N1), .Z(N3) );
  GTECH_BUF B_0 ( .A(RST), .Z(N0) );
  GTECH_BUF B_1 ( .A(N2), .Z(N1) );
  GTECH_NOT I_0 ( .A(RST), .Z(N2) );
endmodule


module comb ( A, B, R );
  input A, B;
  output R;


  GTECH_AND2 C7 ( .A(A), .B(B), .Z(R) );
endmodule


module top ( CLK, RST, A, B, R );
  input CLK, RST, A, B;
  output R;
  wire   q_ff1, q_ff2, r_comb;

  ff uu_ff_in1 ( .CLK(CLK), .RST(RST), .D(A), .Q(q_ff1) );
  ff uu_ff_in2 ( .CLK(CLK), .RST(RST), .D(B), .Q(q_ff2) );
  comb uu_comb ( .A(q_ff1), .B(q_ff2), .R(r_comb) );
  ff uu_ff_out ( .CLK(CLK), .RST(RST), .D(r_comb), .Q(R) );
endmodule
```

VLSI Flow / GTECH

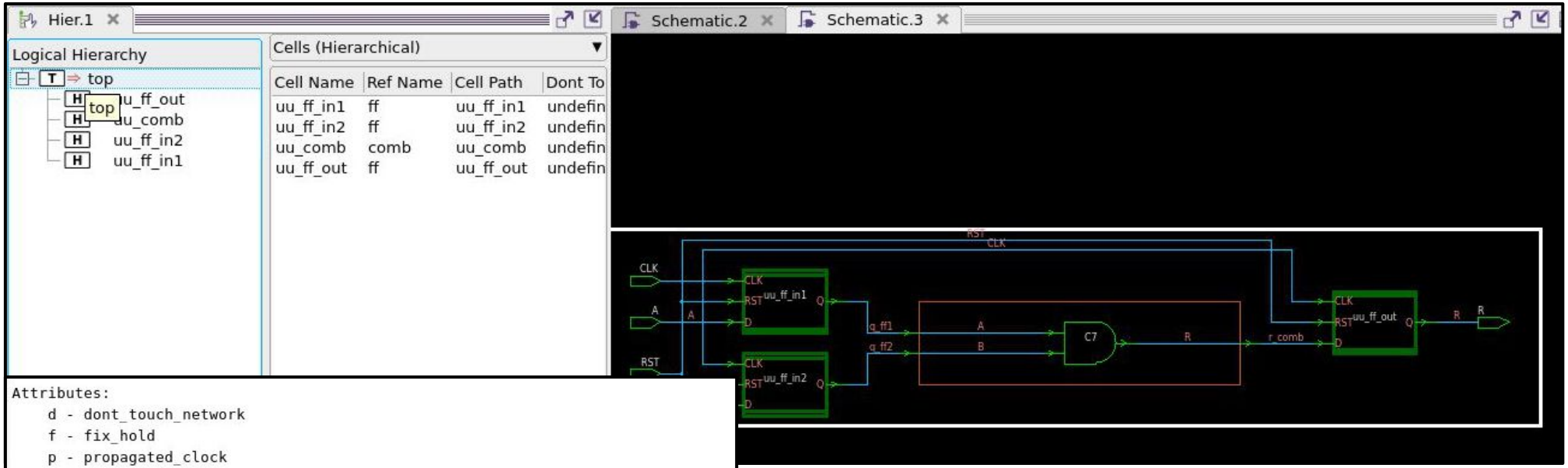


VLSI Flow / GTECH



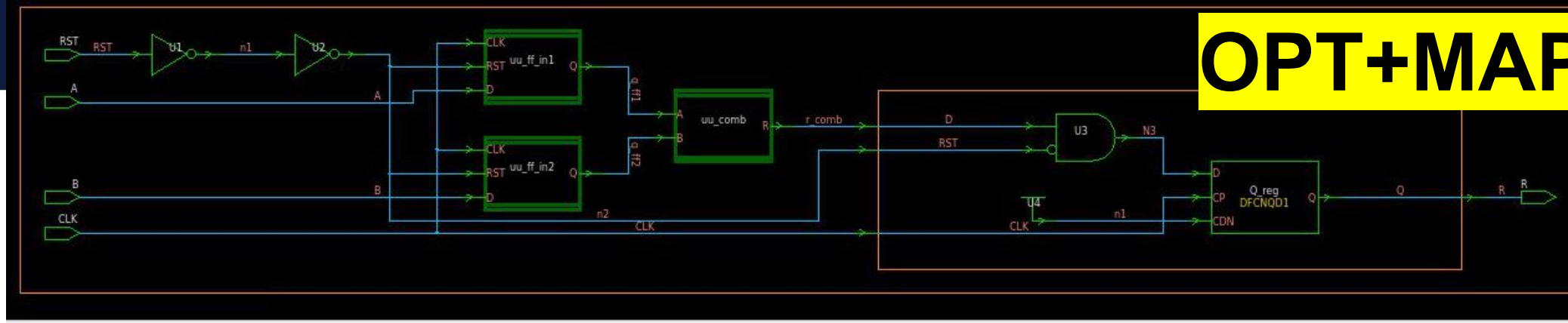


VLSI Flow

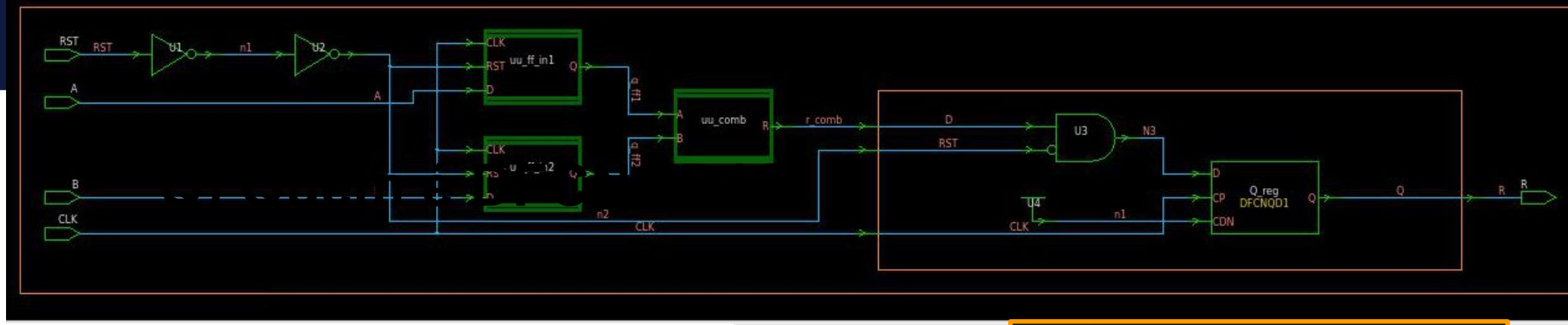



```
Attributes:
    d - dont_touch_network
    f - fix_hold
    p - propagated_clock
    G - generated_clock
    g - lib_generated_clock

Clock          Period   Waveform            Attrs     Sources
------------------------------------------------------------------------------
clk             20.00   {0 10}                        {CLK}
------------------------------------------------------------------------------
1
Current design is 'top'.
dc_shell>
```

Podemos mapear e optimizar para o PDK

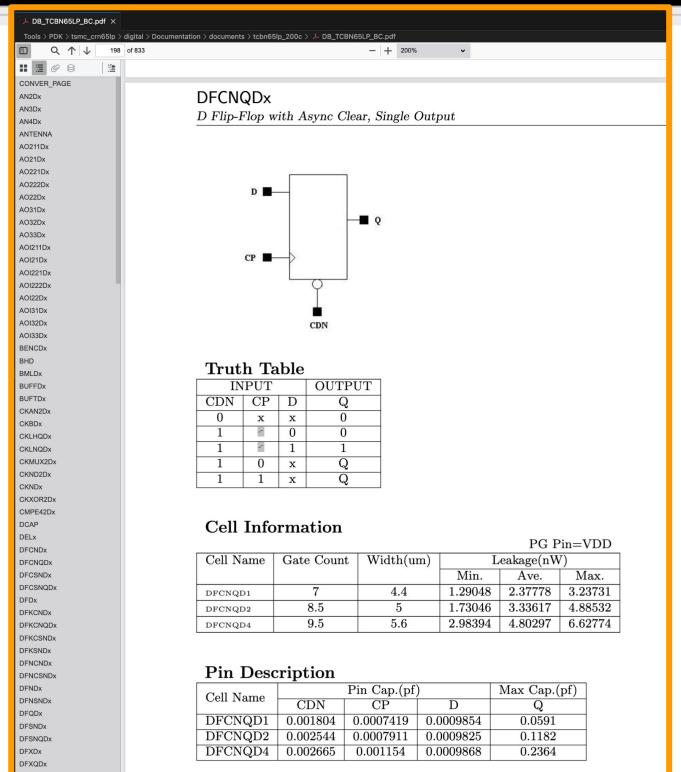


Q) Quem é DFCNQD1?



Q) Quem é DFCNQD1?

R: É um flip-flop tipo D da *Standard Cell*



DFCNQDx

D Flip-Flop with Async Clear, Single Output

Truth Table

INPUT			OUTPUT
CDN	CP	D	Q
0	x	x	0
1	↑	0	0
1	↑	1	1
1	0	x	Q
1	1	x	Q

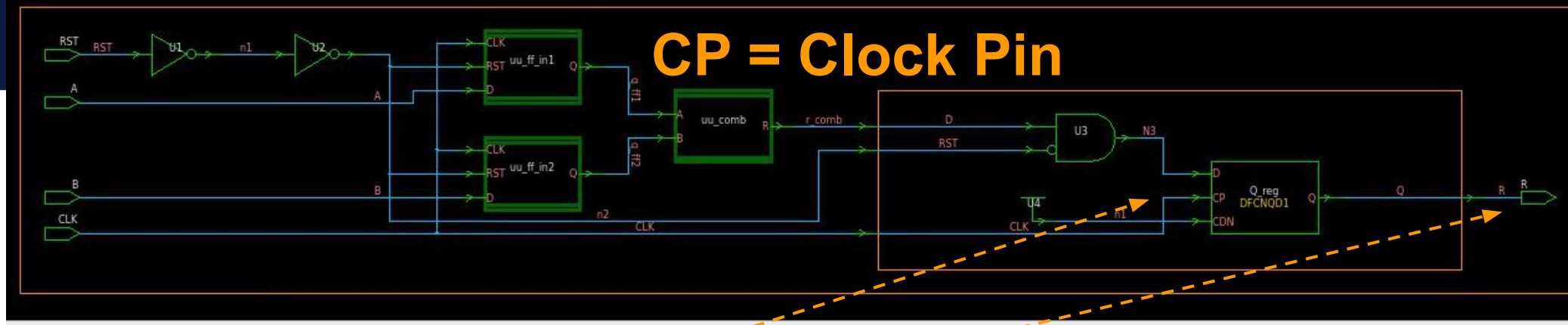
Cell Information

PG Pin=VDD

Cell Name	Gate Count	Width(um)	Leakage(nW)		
			Min.	Ave.	Max.
DFCNQD1	7	4.4	1.29048	2.37778	3.23731
DFCNQD2	8.5	5	1.73046	3.33617	4.88532
DFCNQD4	9.5	5.6	2.98394	4.80297	6.62774

Pin Description

Cell Name	Pin Cap.(pf)			Max Cap.(pf)
	CDN	CP	D	Q
DFCNQD1	0.001804	0.0007419	0.0009854	0.0591
DFCNQD2	0.002544	0.0007911	0.0009825	0.1182
DFCNQD4	0.002665	0.001154	0.0009868	0.2364




```
Point                                   Incr       Path
-----------------------------------------------------------
clock clk (rise edge)                   0.00       0.00
clock network delay (ideal)             1.66       1.66
uu_ff_out/Q_reg/CP (DFCNQD1)            0.00       1.66 r
uu_ff_out/Q_reg/Q (DFCNQD1)             0.61       2.27 r
uu_ff_out/Q (ff_1)                      0.00       2.27 r
R (out)                                 0.00       2.27 r
data arrival time                                  2.27

clock clk (rise edge)                  20.00      20.00
clock network delay (ideal)             1.66      21.66
clock uncertainty                      -2.00      19.66
output external delay                 -10.00       9.66
data required time                                 9.66
-----------------------------------------------------------
data required time                                 9.66
data arrival time                                 -2.27
-----------------------------------------------------------
slack (MET)                                        7.39
```

Static Timing Analysis (STA)

dc_cons_setup.tcl M

prj > 0x01-basic-vlsi-synth > dc-scripts > dc_cons_setup.tcl

```
# ------------------------------------------------------------------------------
# Constraints Setup and Pre-Compiled Reports
# ------------------------------------------------------------------------------

# create clock
create_clock -period $clk_val [get_ports CLK] -name clk

# characteristics of clk
set_clock_uncertainty     -setup [expr $clk_val*0.1000] [get_clocks clk]
set_clock_transition      -max   [expr $clk_val*0.1000] [get_clocks clk]
set_clock_latency -source -max   [expr $clk_val*0.0500] [get_clocks clk]
set_clock_latency         -max   [expr $clk_val*0.0328] [get_clocks clk]

# input and output delay
set_input_delay  -max [expr $clk_val*0.4] -clock clk [remove_from_collection [all_inputs] [get_ports CLK]]
set_output_delay -max [expr $clk_val*0.5] -clock clk [all_outputs]
```

report_timing => critical path

GTECH => LOGICAL NETLIST

gtech_unmapped.v

prj > 0x01-basic-vlsi-synth > dc-scripts > outputs_top > gtech_unmapped.v

```
1  ////////////////////////////////////////////////////////////
2  // Created by: Synopsys Design Compiler(R)
3  // Version   : X-2025.06-SP4
4  // Date      : Mon Apr  6 17:33:01 2026
5  ////////////////////////////////////////////////////////////
6
7
8  module ff ( CLK, RST, D, Q );
9    input CLK, RST, D;
10   output Q;
11   wire   N0, N1, N2, N3;
12
13   \**SEQGEN**  Q_reg ( .clear(1'b0), .preset(1'b0), .next_state(N3),
14         .clocked_on(CLK), .data_in(1'b0), .enable(1'b0), .Q(Q), .synch_clear(
15        1'b0), .synch_preset(1'b0), .synch_toggle(1'b0), .synch_enable(1'b1)
16         );
17   SELECT_OP C11 ( .DATA1(1'b0), .DATA2(D), .CONTROL1(N0), .CONTROL2(N1), .Z(N3) );
18   GTECH_BUF B_0 ( .A(RST), .Z(N0) );
19   GTECH_BUF B_1 ( .A(N2), .Z(N1) );
20   GTECH_NOT I_0 ( .A(RST), .Z(N2) );
21 endmodule
22
23
24 module comb ( A, B, R );
25   input A, B;
26   output R;
27
28
29   GTECH_AND2 C7 ( .A(A), .B(B), .Z(R) );
30 endmodule
31
32
33 module top ( CLK, RST, A, B, R );
34   input CLK, RST, A, B;
35   output R;
36   wire   q_ff1, q_ff2, r_comb;
37
38   ff uu_ff_in1 ( .CLK(CLK), .RST(RST), .D(A), .Q(q_ff1) );
39   ff uu_ff_in2 ( .CLK(CLK), .RST(RST), .D(B), .Q(q_ff2) );
40   comb uu_comb ( .A(q_ff1), .B(q_ff2), .R(r_comb) );
41   ff uu_ff_out ( .CLK(CLK), .RST(RST), .D(r_comb), .Q(R) );
42 endmodule
```

top.v

prj > 0x01-basic-vlsi-synth > dc-scripts > outputs_top > top.v

```
1  ////////////////////////////////////////////////////////////
2  // Created by: Synopsys DC Ultra(TM) in wire load mode
3  // Version   : X-2025.06-SP4
4  // Date      : Mon Apr  6 17:14:11 2026
5  ////////////////////////////////////////////////////////////
6
7
8  module ff_0 ( CLK, RST, D, Q );
9    input CLK, RST, D;
10   output Q;
11   wire   n1;
12
13   DFKCNQD1 Q_reg ( .CN(n1), .D(D), .CP(CLK), .Q(Q) );
14   CKND0 U3 ( .I(RST), .ZN(n1) );
15 endmodule
16
17
18 module ff_2 ( CLK, RST, D, Q );
19   input CLK, RST, D;
20   output Q;
21   wire   N3, n1;
22
23   INR2D0 U3 ( .A1(D), .B1(RST), .ZN(N3) );
24   DFCNQD1 Q_reg ( .D(N3), .CP(CLK), .CDN(n1), .Q(Q) );
25   TIEH U4 ( .Z(n1) );
26 endmodule
27
28
29 module comb ( A, B, R );
30   input A, B;
31   output R;
32
33
34   CKAN2D0 U1 ( .A1(A), .A2(B), .Z(R) );
35 endmodule
36
37
38 module ff_1 ( CLK, RST, D, Q );
39   input CLK, RST, D;
40   output Q;
41   wire   N3, n1;
42
43   INR2D0 U3 ( .A1(D), .B1(RST), .ZN(N3) );
44   DFCNQD1 Q_reg ( .D(N3), .CP(CLK), .CDN(n1), .Q(Q) );
45   TIEH U4 ( .Z(n1) );
46 endmodule
47
48
49 module top ( CLK, RST, A, B, R );
50   input CLK, RST, A, B;
51   output R;
52   wire   q_ff1, q_ff2, r_comb, n1, n2;
53
54   ff_0 uu_ff_in1 ( .CLK(CLK), .RST(n2), .D(A), .Q(q_ff1) );
55   ff_2 uu_ff_in2 ( .CLK(CLK), .RST(n2), .D(B), .Q(q_ff2) );
56   comb uu_comb ( .A(q_ff1), .B(q_ff2), .R(r_comb) );
57   ff_1 uu_ff_out ( .CLK(CLK), .RST(n2), .D(r_comb), .Q(R) );
58   CKND0 U1 ( .I(RST), .ZN(n1) );
59   CKND0 U2 ( .I(n1), .ZN(n2) );
60 endmodule
```

RTL => GTECH => LOGICAL NETLIST

ff.sv

```
module ff(
  input  logic CLK,
  input  logic RST,
  input  logic D,
  output logic Q
);

  always_ff @(posedge CLK) begin
    if(RST)
      Q <= 1'b0;
    else
      Q <= D;
  end

endmodule
```

top.sv

```
module top(
  input  logic CLK,
  input  logic RST,
  input  logic A,
  input  logic B,
  output logic R
);
  // "wires""
  logic q_ff1, q_ff2, r_comb;

  // FF input 1
  ff uu_ff_in1(
    .CLK(CLK  ),
    .RST(RST  ),
    .D  (A    ),
    .Q  (q_ff1)
  );

  // FF input 2
  ff uu_ff_in2(
    .CLK(CLK  ),
    .RST(RST  ),
    .D  (B    ),
    .Q  (q_ff2)
  );

  // FF
  comb uu_comb(
    .A(q_ff1 ),
    .B(q_ff2 ),
    .R(r_comb)
  );

  ff uu_ff_out(
    .CLK(CLK   ),
    .RST(RST   ),
    .D  (r_comb),
    .Q  (R     )
  );
endmodule
```

comb.sv

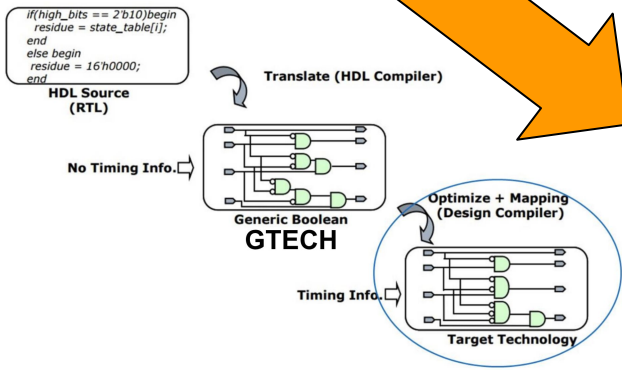
```
module comb(
  input  logic A,
  input  logic B,
  output logic R
);

  always_comb begin
    R = A & B;
  end

endmodule
```

gtech_unmapped.v

```
////////////////////////////////////////////////////
// Created by: Synopsys Design Compiler(
// Version   : X-2025.06-SP4
// Date      : Mon Apr  6 17:33:01 2026
////////////////////////////////////////////////////


module ff ( CLK, RST, D, Q );
  input CLK, RST, D;
  output Q;
  wire   N0, N1, N2, N3;

  \**SEQGEN**  Q_reg ( .clear(1'b0), .preset(1'b0), .next_state(N3),
        .clocked_on(CLK), .data_in(1'b0), .enable(1'b0), .Q(Q), .synch_c
        1'b0), .synch_preset(1'b0), .synch_toggle(1'b0), .synch_enable(1
        );
  SELECT_OP C11 ( .DATA1(1'b0), .DATA2(D), .CONTROL1(N0), .CONTROL2(N1),
  GTECH_BUF B_0 ( .A(RST), .Z(N0) );
  GTECH_BUF B_1 ( .A(N2), .Z(N1) );
  GTECH_NOT I_0 ( .A(RST), .Z(N2) );
endmodule


module comb ( A, B, R );
  input A, B;
  output R;


  GTECH_AND2 C7 ( .A(A), .B(B), .Z(R) );
endmodule


module top ( CLK, RST, A, B, R );
  input CLK, RST, A, B;
  output R;
  wire   q_ff1, q_ff2, r_comb;

  ff uu_ff_in1 ( .CLK(CLK), .RST(RST), .D(A), .Q(q_ff1) );
  ff uu_ff_in2 ( .CLK(CLK), .RST(RST), .D(B), .Q(q_ff2) );
  comb uu_comb ( .A(q_ff1), .B(q_ff2), .R(r_comb) );
  ff uu_ff_out ( .CLK(CLK), .RST(RST), .D(r_comb), .Q(R) );
endmodule
```

top.v

```
////////////////////////////////////////////////////////////////////
// Created by: Synopsys DC Ultra(TM) in wire load mode
// Version   : X-2025.06-SP4
// Date      : Mon Apr  6 17:14:11 2026
////////////////////////////////////////////////////////////////////


module ff_0 ( CLK, RST, D, Q );
  input CLK, RST, D;
  output Q;
  wire   n1;

  DFKCNQD1 Q_reg ( .CN(n1), .D(D), .CP(CLK), .Q(Q) );
  CKND0 U3 ( .I(RST), .ZN(n1) );
endmodule


module ff_2 ( CLK, RST, D, Q );
  input CLK, RST, D;
  output Q;
  wire   N3, n1;

  INR2D0 U3 ( .A1(D), .B1(RST), .ZN(N3) );
  DFCNQD1 Q_reg ( .D(N3), .CP(CLK), .CDN(n1), .Q(Q) );
  TIEH U4 ( .Z(n1) );
endmodule


module comb ( A, B, R );
  input A, B;
  output R;


  CKAN2D0 U1 ( .A1(A), .A2(B), .Z(R) );
endmodule


module ff_1 ( CLK, RST, D, Q );
  input CLK, RST, D;
  output Q;
  wire   N3, n1;

  INR2D0 U3 ( .A1(D), .B1(RST), .ZN(N3) );
  DFCNQD1 Q_reg ( .D(N3), .CP(CLK), .CDN(n1), .Q(Q) );
  TIEH U4 ( .Z(n1) );
endmodule


module top ( CLK, RST, A, B, R );
  input CLK, RST, A, B;
  output R;
  wire   q_ff1, q_ff2, r_comb, n1, n2;

  ff_0 uu_ff_in1 ( .CLK(CLK), .RST(n2), .D(A), .Q(q_ff1) );
  ff_2 uu_ff_in2 ( .CLK(CLK), .RST(n2), .D(B), .Q(q_ff2) );
  comb uu_comb ( .A(q_ff1), .B(q_ff2), .R(r_comb) );
  ff_1 uu_ff_out ( .CLK(CLK), .RST(n2), .D(r_comb), .Q(R) );
  CKND0 U1 ( .I(RST), .ZN(n1) );
  CKND0 U2 ( .I(n1), .ZN(n2) );
```

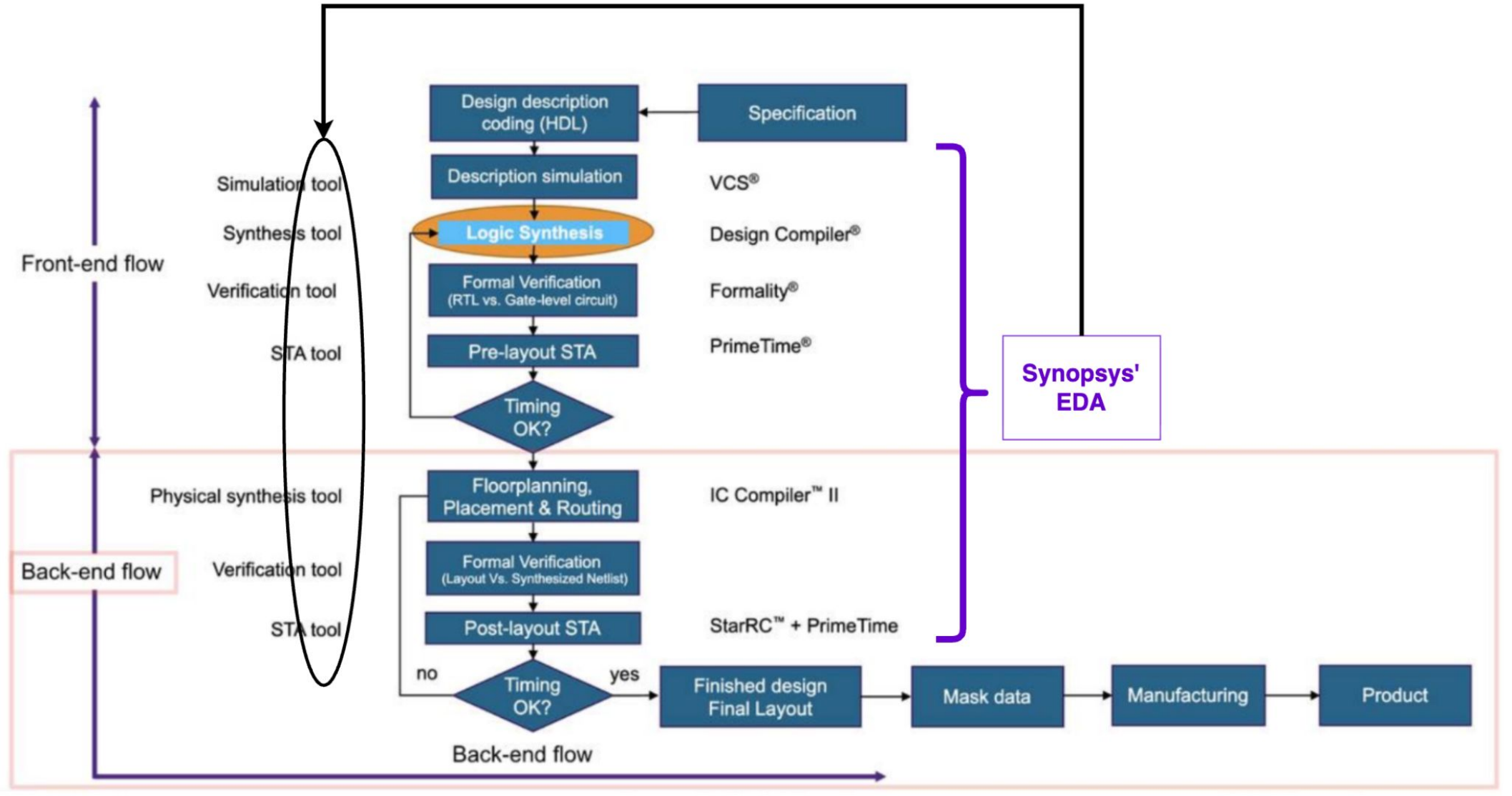


Acabamos de fazer isso na prática…

Isso é o básico do Front-end…

Back-end

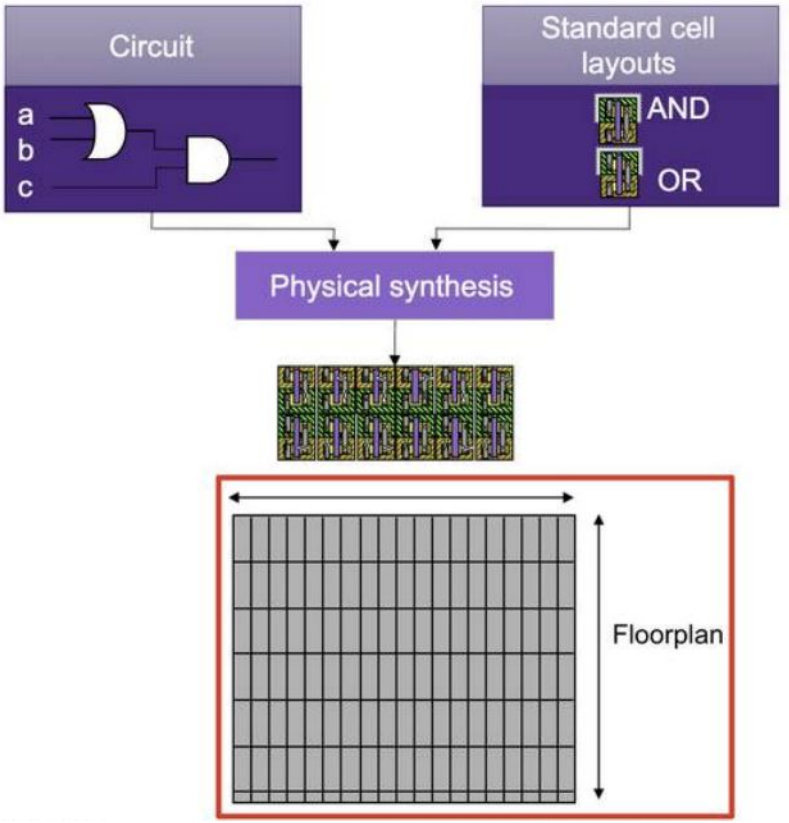
EDAs are tools to run the VLSI flow



VLSI Flow / Back-End

- Continua o processo de "desenhar" sendo que com mais detalhes

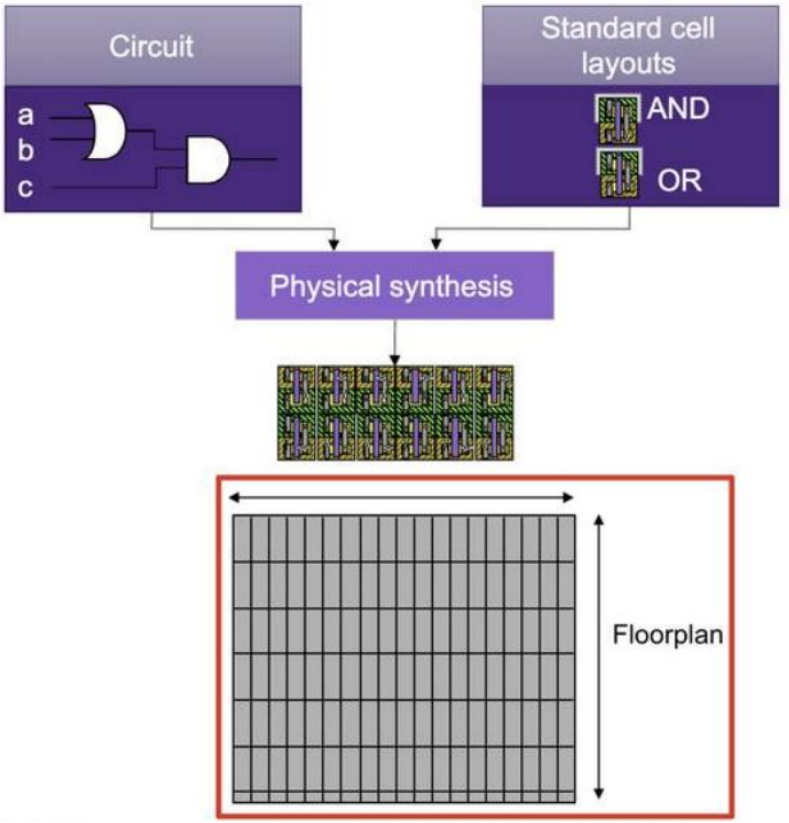
- ***1) Floorplan***



VLSI Flow / Back-End

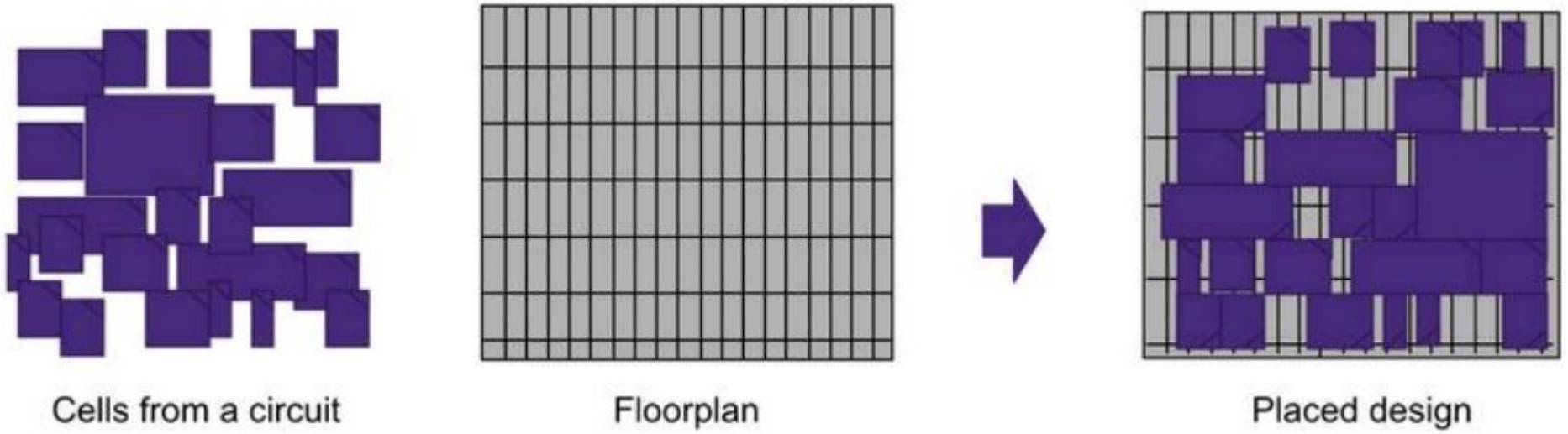
- Continua o processo de "desenhar" sendo que com mais detalhes

- ***1) Floorplan***



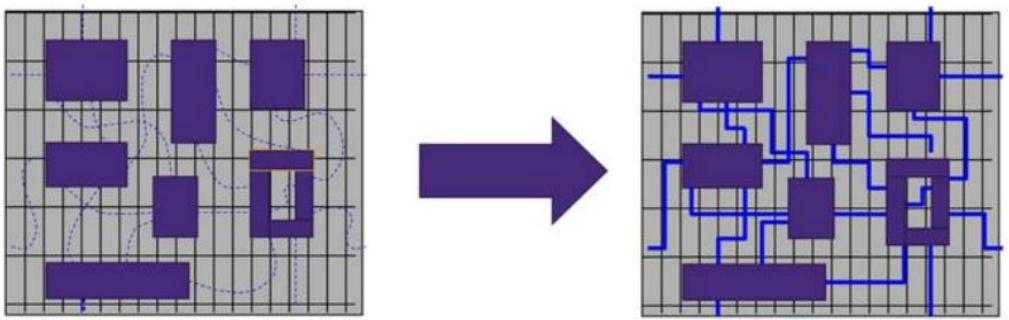
VLSI Flow / Back-End

2) Cells a nívels de layout são ***PLACED***



VLSI Flow / Back-End

3) Tudo isso é conectado ***(ROUTING)***



VLSI Flow / Back-End

3) A conexão do clock é uma etapa a parte chamada **Clock Tree Synthesis** (CTS)

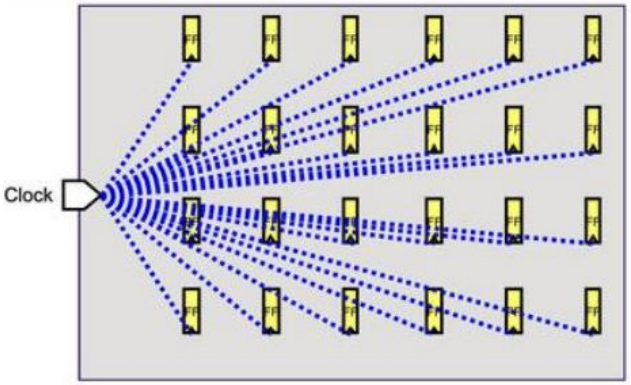


Figure 1

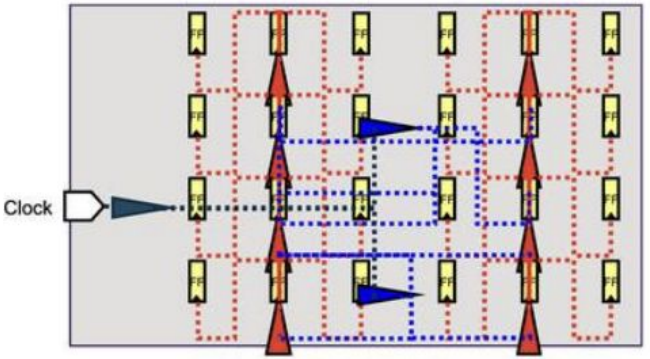
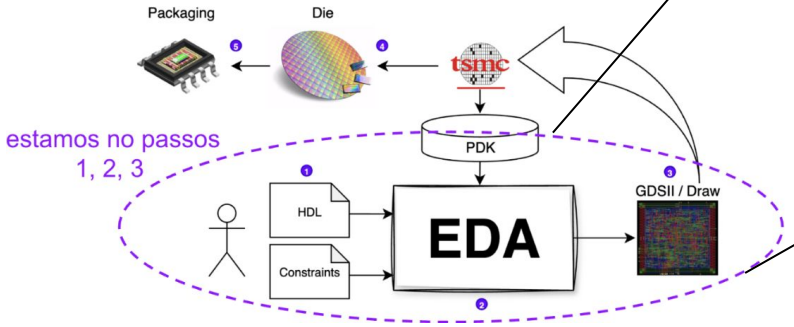
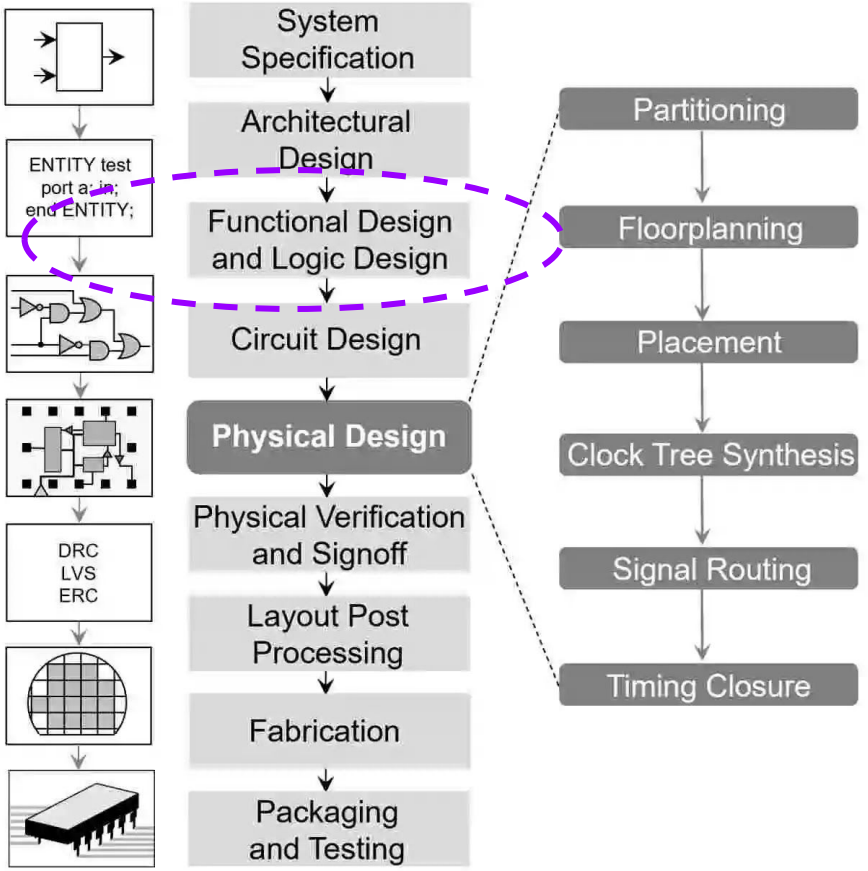


Figure 2

VLSI Flow / Back-End

Em resumo:



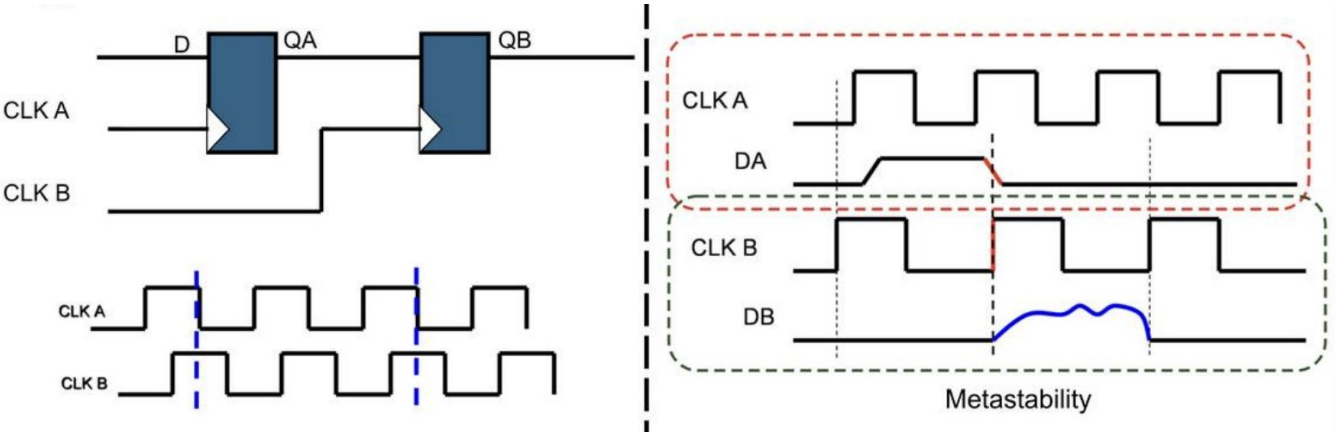


Continuação

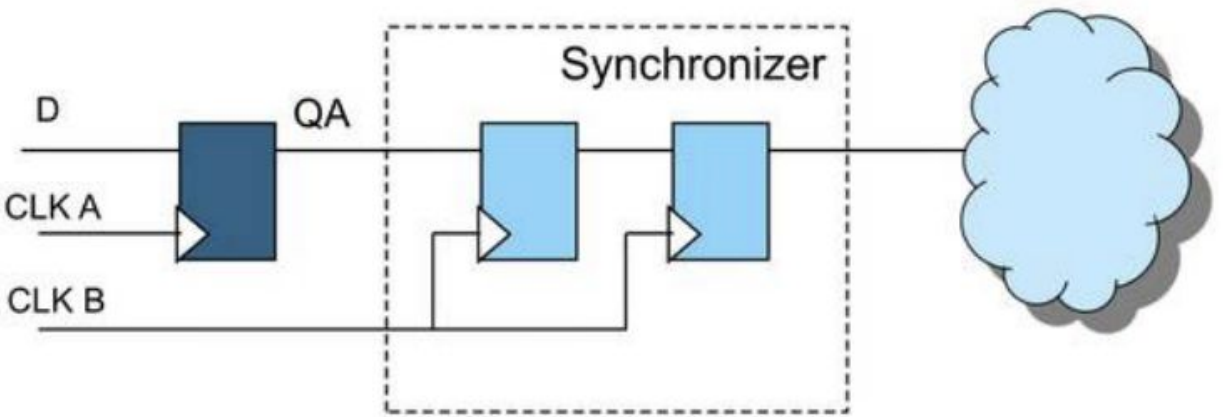
=> VSDM / Digital Fundametals / ASIC Design Flow

- **VDSM**
  - CDC

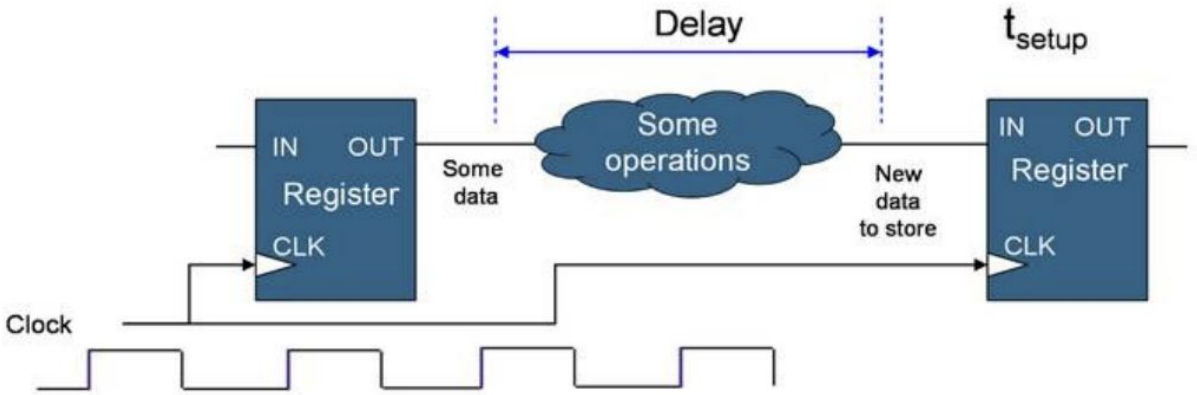
problema:



uma das soluções:

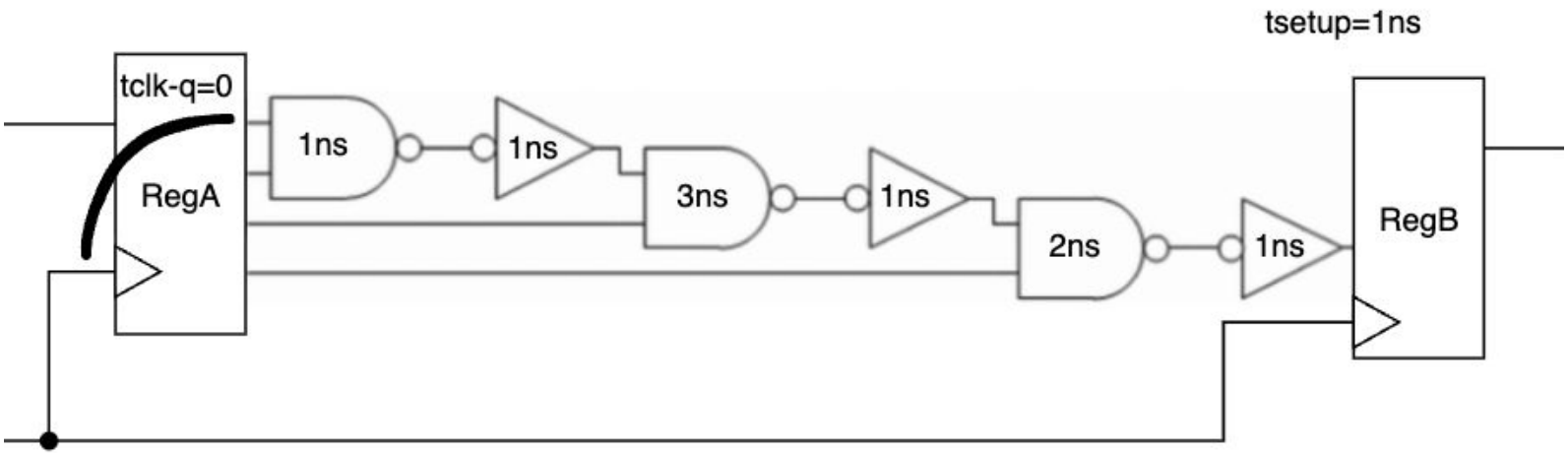


- **VDSM**
  - Timing Analysys
  - **STA**
    - setup/hold



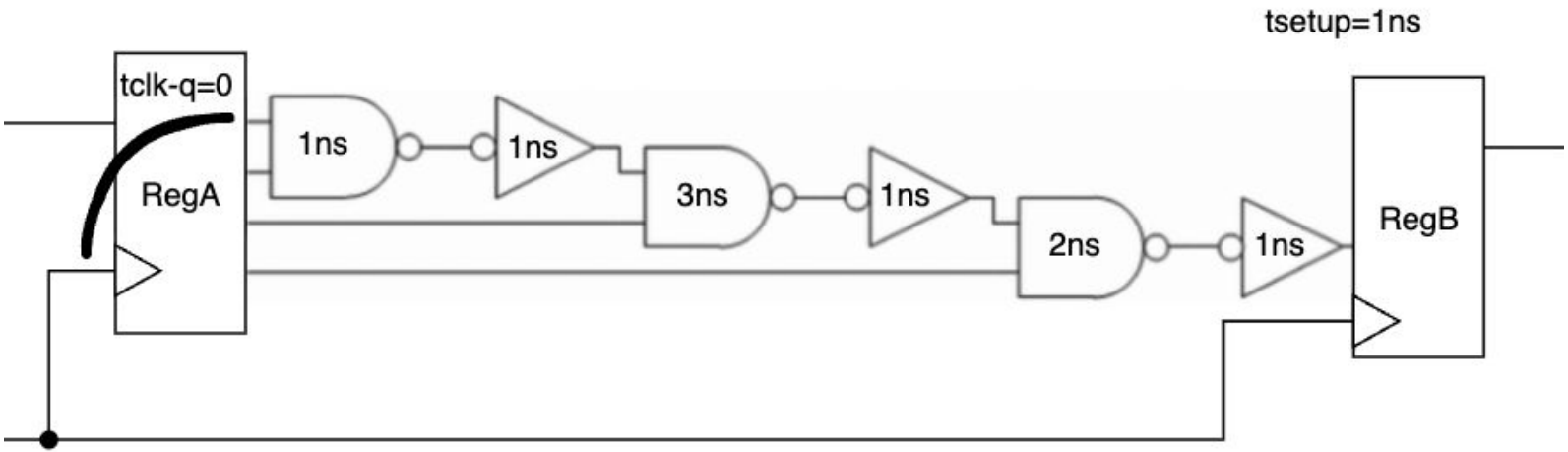
- EXERCÍCIO **"tempo crítico + clock"**
  - Considerando propagação de nets=0 e clock ideal (0: skew/jitter)
  - Tsetup=1ns
  - **Q1) Quantos flops temos aqui?**
  - **Q2) Qual a freq. máxima permitida p/ esse circuito?**

$$P_{clk} > t_{clk2q} + t_{pd} + t_{setup}$$



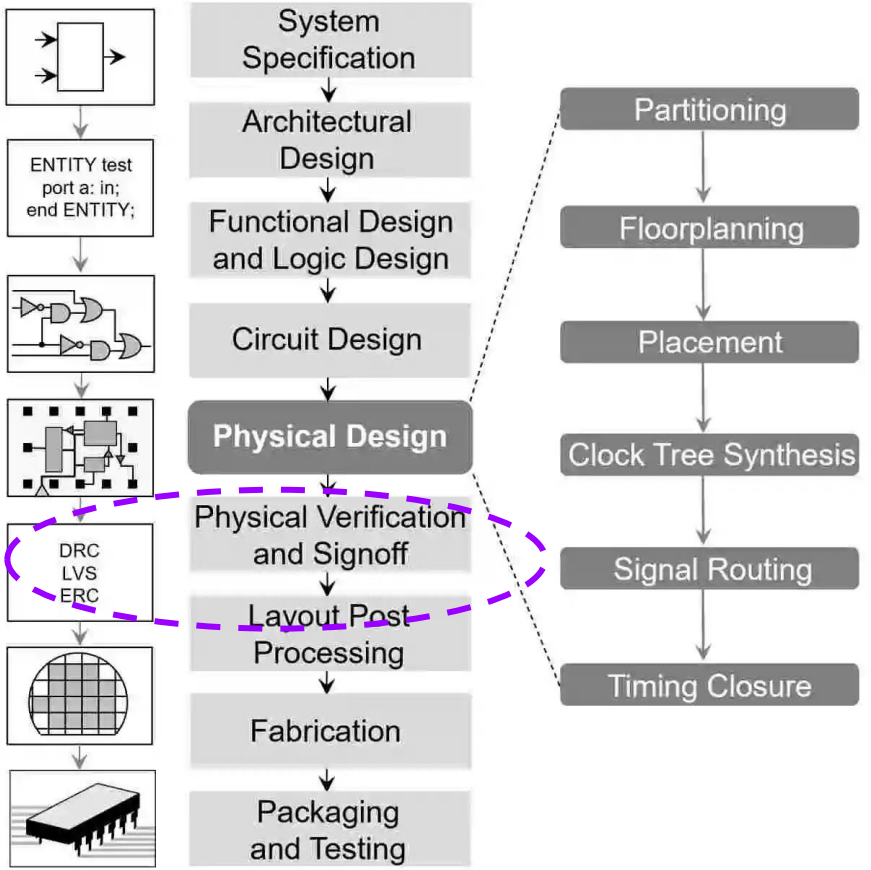
- EXERCÍCIO **"tempo crítico + clock"**
  - Considerando propagação de nets=0 e clock ideal (0: skew/jitter)
  - Tsetup=1ns
  - **Q1) Quantos flops temos aqui?** 5
  - **Q2) Qual a freq. máxima permitida p/ esse circuito?** 100MHz

$$P_{clk} > t_{clk2q} + t_{pd} + t_{setup}$$



=> VSDM / Digital Fundametals / ASIC Design Flow

- **VDSM - outros tópicos**
  - Electromigration
  - Antenna Effect
  - Filler/Corners Cells
  - ...



=> VSDM / **Digital Fundametals** / **ASIC Design Flow**

- Conversão:
  - bin2dec;
  - dec2bin
  - hex2bin;
  - bin2hex;
  - hex2dec;
  - dec2hex;

- SOP

sum of products

- POS

product of sums

Mapa de Karnought

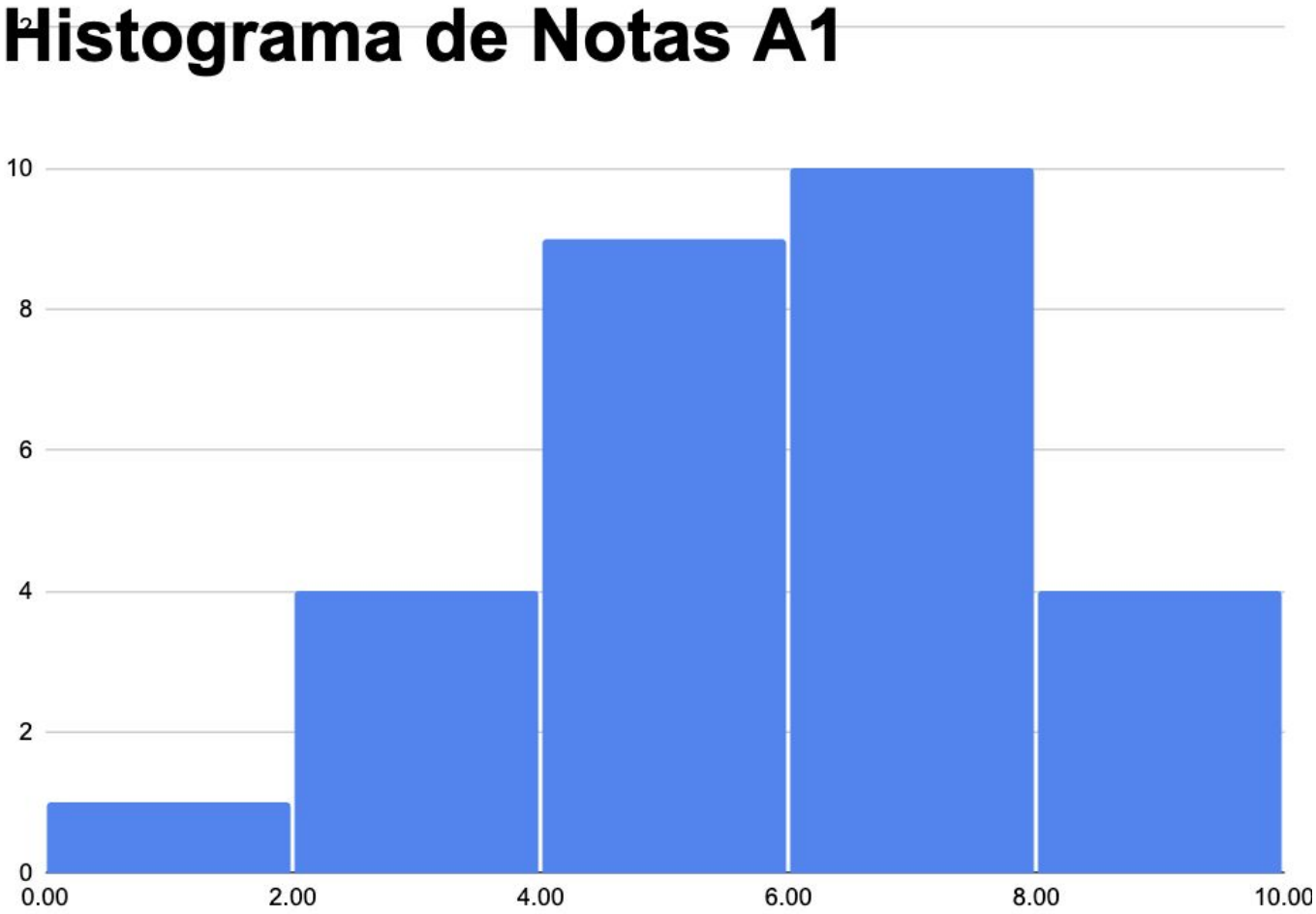
não será avaliado no exame

- **Circuitos Combinacionais**
  - Expressão booleana
- **Circuitos Sequencias**
  - Igual da sala em **SystemVlog**

FSM

não será avaliado no exame

Prova 01



TKS


EXECUÇÃO:
iRede
UnB
UFRGS
UNIFEI
Universidade Federal do Ceará
APOIO:
synopsys
COORDENAÇÃO:
Softex
INICIATIVA:
CHIP TECH BRASIL
MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO DO BRASIL